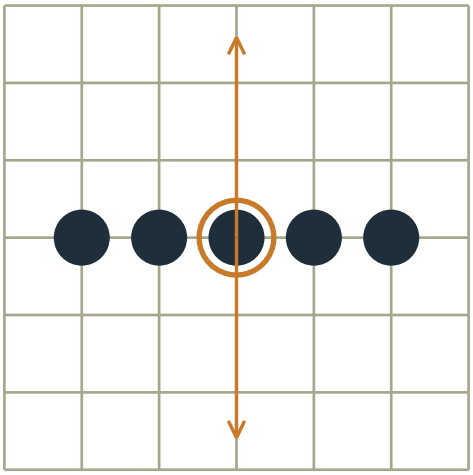
READ THE CODE, UNDERSTAND THE GAME

Gomoku3

C++ 初学者
源码阅读伴侣

打开源码，跟着一盘棋走。

从一次鼠标点击，到棋盘判胜；从保存一局棋，到电脑思考与两台电脑同步。按依赖关系阅读，逐个认识函数、结构体和变量。



从最新落点往两端数
左侧 2 + 自己 1 + 右侧 2 = 5

(dr, dc) = (0, 1) 为水平
(1, 0) 为竖直；另有两条斜线

row 向下增大；col 向右增大
棋盘下标均从 0 到 14

图：中心金圈是最新一手。箭头示意另一条待检查的竖线；四条线逐一检查，任一方向达到 5 即获胜。

源码核对：2026-10-09 | 范围：Gomoku3 当前运行源码

已验证：客户端与服务端编译成功；4 组 C++ 测试通过。
本书不收录整份源码。文件名与行号用于在编辑器中定位。

阅读地图 / 可点击目录

01 先认清你正在读的程序 3

02 读这份代码需要的 C++ 5

03 棋盘：从数据到判胜 8

04 界面：一次点击怎样成为一手棋 11

05 存档：保存历史，读回时重新验证 19

06 复盘：播放历史，不改原棋局 23

07 AI：先找好点，再假设双方都认真下 26

08 线程：让电脑思考时，窗口还能响应 33

09 联网规则：两个人共享同一个裁判 36

10 TCP：收到的是字节，不是一条条消息 41

11 服务端：一个循环，管理两个人的房间 46

12 联网界面：把状态变成可操作的页面 50

13 用七个练习，把源码变成自己的知识 59

附录 A 查阅表、容易误读的细节与验证记录 61

附录 B 函数检索表 67

建议：第一轮按章读主线，第二轮再逐卡查变量。书签可展开章节，末尾函数检索表可按名字定位。

CHAPTER 01

先认清你正在读的程序

先建立整体地图，再钻进代码。不要从 main.cpp 第一行开始逐字硬啃。

本书只依据工作区 Gomoku3 的当前源码。Gomoku、全AI五子棋、材料里的示例和 tests 下的历史运行副本都不属于本书的运行源码范围。tests 是验证材料，不会被链接进游戏。

实际编译单元	作用
main.cpp → Gomoku.exe	图形客户端，包含其余模块；单文件编译通常输出 main.exe。
server_main.cpp → GomokuServer.exe	独立控制台程序；房主通过隐藏子进程启动它。它有自己的 main。
8 个 .h 文件	不仅声明类型，也定义很多函数；经 #include 展开后一起编译。Compile=0 不等于“不参与程序”。
build.ps1 / 两个 .dev	确认编译命令和链接库；两个 main 不能混在同一次链接中。

证据：build.ps1 分两次调用 g++；Gomoku.dev 仅 main.cpp 的 Compile=1；GomokuServer.dev 仅 server_main.cpp 的 Compile=1。当前 10 个运行源码文件共 2608 行（含注释、空行）。旧 exe 的修改时间早于部分源码，不能据其文件名断言它与当前源码一致。

图 1 模块依赖与阅读主线



如何打开源码

用编辑器打开 Gomoku3 工程；C++ 源文件按 GBK / CP936 读取，README 为 UTF-8。若中文注释变成乱码，先重新选择编码，不要直接保存覆盖。行号来自此次源码快照；日后编辑导致行号变化时，搜索同名函数即可。

源码中的 \uXXXX 是 Unicode 转义；例如宽字符串 L"..." 配合 Windows 的 W 结尾接口显示中文。源文件编码、内存字符串编码、存档 UTF-8 编码是三个不同层次。

推荐分 7 次消化

顺序 / 建议时长	读完应能讲清楚
1 / 45-60 分钟	第 02-03 章：棋盘如何存、如何判胜、如何悔棋。
2 / 60-90 分钟	第 04 章：页面、弹窗和点击事件如何分工。
3 / 45-60 分钟	第 05-06 章：读档为何要重放；复盘如何暂停。
4 / 60-90 分钟	第 07 章：候选点、评分和递归搜索。
5 / 30-45 分钟	第 08 章：线程快照、取消标志与 generation。
6 / 60-90 分钟	第 09-10 章：消息合法性、TCP 分帧。
7 / 60-90 分钟	第 11-12 章：服务端循环、联网 UI 与退出流程。

时长仅供参考；陌生 API 先看输入、输出和副作用，再回查变量表。

CHAPTER 02

读这份代码需要的 C++

每个语法点，都对应你马上会遇到的一行代码。

语法 / 类型	在本工程里怎样理解
struct / class	把数据和操作放在一起。struct 默认 public；class 默认 private。AISearch 的 public: 以前是内部实现。
对象与成员访问	LocalGame game 创建对象；game.turn 访问成员；self->answer 通过指针访问成员。
构造函数	名字与类型相同，无返回类型。LocalGame() 自动调用 reset()，并不是每次落子都重新构造。
初始化列表 :	AIEntry():key(0)... 在进入函数体前初始化成员；初始化顺序由成员声明顺序决定。
const T&	只读引用：不复制整个对象，函数也不能通过此引用修改对象。
T& 输出参数	take(AIPoint& point,int& gen) 可把结果写回调用者的变量；返回 bool 另外表示是否成功。
const 成员函数	wins(...) const 不改当前对象的成员；它仍可以使用局部变量 count。
指针 * 与取地址 &	volatile LONG* cancel 指向共享标志；&cancelled 取得该变量地址。NULL 表示没有对象/句柄。
vector<T>	可增长数组；push_back 追加、back 看末尾、pop_back 删除末尾、clear 清空。下标从 0 开始。
string / wstring	字节串与宽字符串。Windows 下 wchar_t 通常为 UTF-16 代码单元；24 个代码单元不严格等于 24 个 Unicode 字符。
size_t	容器长度/索引常用的无符号整数。size()-1 在空容器时可能下溢，所以倒序搜索先转 int。
枚举 enum	给整数起名字。Page、Dialog、NetPhase 各管不同状态，不能混为一个编号空间。
|| 与 &&	从左到右短路求值。place 先判越界，后访问 cells，避免越界读取。

语法 / 类型	在本工程里怎样理解
?: / !	条件 ? 值1 : 值2；! 表示逻辑取反。!result 意为结果为 0，即未结束。
static / inline	static 成员函数无 this，可给线程当入口；局部 static 只初始化一次。inline 也解决头文件同一定义的链接问题，不保证机器码必定内联。
lambda [](...){}	这里用于排序。candidates 的 a.score>b.score 表示高分排在前面。
static_cast / reinterpret_cast	前者进行显式类型转换；后者把地址/句柄按另一种类型解释，常见于 Win32 接口。
位操作 ^ << >> |	^ 异或用于随机数和哈希；<<、>> 在整数上可表示移位，在流上表示输入输出。
HANDLE / HWND / SOCKET	系统资源的标识，不能把它们当普通 C++ 对象随意 delete；分别用对应的关闭函数。

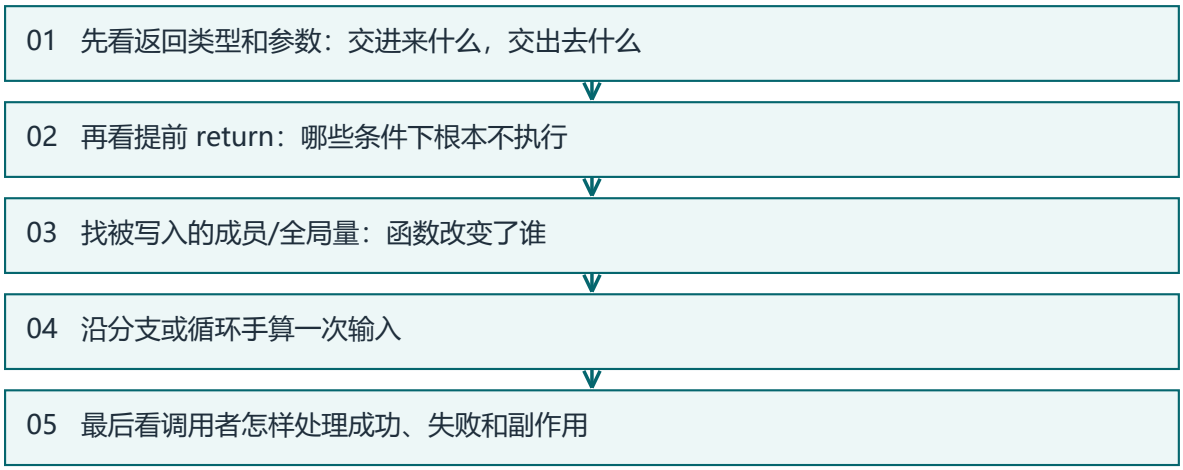
三个“返回值”要分开

place 返回 true 表示“这一步成功放下”，不是“赢了”；是否赢要读 game.result。parseRecord 返回 true 表示校验通过，记录从 output 引用取。线程函数 run 返回 0 表示线程退出，其计算答案写在 answer 中。

作用域与同名变量

main.cpp 的全局 game 是界面当前棋局；AISearch::board 是搜索副本；ReplayPlayer::board 是播放到当前步的展示盘；NetState::game 是网络权威状态中的棋盘。它们类型相同，但不是同一块数据。局部 r 在读档里可能是 GameRecord，在棋盘循环里却是行号，必须连同所在函数读。

图 2 读函数的固定顺序



小练习

为什么 3 - turn 能切换黑白？答：turn 只允许 1、2；3-1=2，3-2=1。若 turn=0，这个表达式并不会得到合法棋色，正确性依赖变量的取值约定。

CHAPTER 03

棋盘：从数据到判胜

打开 game_logic.h。这里只管规则，不画图、不联网，也不知道谁是人类。

变量 / 成员	含义与取值
Move.row / Move.col	一手棋的行、列，范围 0..14。
Move.stone	这手的颜色：1 黑，2 白；Move 是一次历史记录。
LocalGame.cells[15][15]	当前盘面：0 空、1 黑、2 白。第一维是行，第二维是列。
LocalGame.turn	下一手轮到谁；未终局时为 1 或 2。终局时不再翻转，保留最后落子方。
LocalGame.result	0 未结束；1 黑胜；2 白胜；3 平局。
LocalGame.moves	按时间顺序保存的 Move 数组，同时支撑悔棋、存档和复盘。

LocalGame::LocalGame

game_logic.h : 21

作用与接口　默认构造空棋局。

执行顺序　直接调用 reset；所有需要空棋盘的地方复用同一初始化逻辑。

LocalGame::reset

game_logic.h : 24

作用与接口　无参数、无返回值；恢复空棋盘。

执行顺序　memset 把 int 数组置零；清空 moves；turn=1，result=0。

参数与局部量　sizeof(cells) 是整块数组的字节数，不是格子个数。

对照源码时留意

只清空 cells 而不清 moves，会使屏幕绘制、手数与存档互相矛盾。

LocalGame::wins

game_logic.h : 32

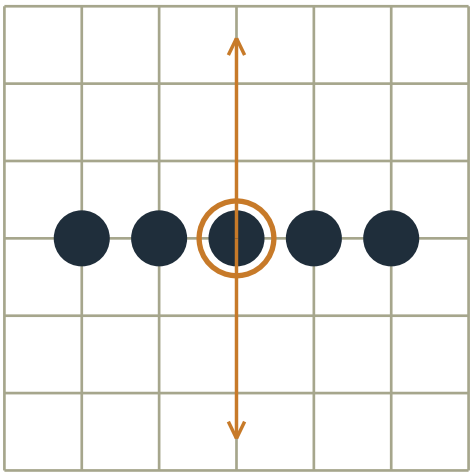
作用与接口 输入最后落点 row、col；返回这一点所在的线上是否有至少五颗同色子。

执行顺序 先取当前格颜色；空格返回 false；对四个方向逐一从中心向两端走，遇到边界、空位或异色停止；累计>=5 返回 true。

参数与局部量 directions[4][2] 是方向增量；d 是方向下标；stone 是本格颜色；count 从 1 算自身；sign=-1/+1 是两端；r、c 是扫描中的坐标。

对照源码时留意

它不检查入口 row/col 是否越界；调用者先保证合法。方向 (1,0) 在行列坐标中是竖直，源码“右”的注释不可照抄。六连也算胜，不含禁手规则。



从最新落点往两端数
左侧 2 + 自己 1 + 右侧 2 = 5

(dr, dc) = (0, 1) 为水平
(1, 0) 为竖直；另有两条斜线

row 向下增大；col 向右增大
棋盘下标均从 0 到 14

图：中心金圈是最新一手。箭头示意另一条待检查的竖线；四条线逐一检查，任一方向达到 5 即获胜。

LocalGame::place

game_logic.h : 54

作用与接口 输入 row、col；true 表示落子成功，false 表示状态没有被正常落子流程更新。

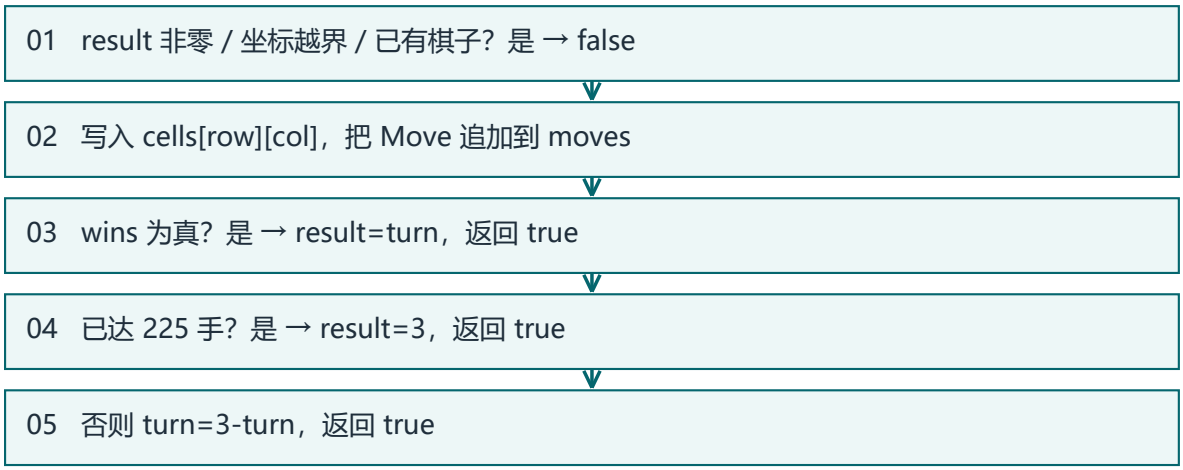
执行顺序 拒绝已结束、越界、占用；写 cells；构造 move 并追加历史；先判胜，再判满盘，最后才交换 turn。

参数与局部量 move 是当前这一步 {row,col,turn}；225=15×15。

对照源码时留意

胜利与满盘同时发生时，先判胜。wins 不会改 result，place 才负责落地终局状态。

图 3 place 的判断顺序



LocalGame::undo

game_logic.h : 66

作用与接口 撤掉最后一手；终局或无历史时返回 false。

执行顺序 复制 moves.back；pop_back 删除历史；将原落点置 0；turn 恢复为这手的颜色。

参数与局部量 move 是被撤回的一手。

对照源码时留意

不支持撤销已经结束的棋局。AI 回合悔棋由上层多次调用这里实现。

手算一局	关键变量变化
刚创建	turn=1，result=0，moves.size()=0
place(7,7)	cells[7][7]=1，moves[0]={7,7,1}，turn=2
再 place(7,7)	返回 false，所有棋局字段不变
place(7,8)	cells[7][8]=2，moves.size()=2，turn=1
undo()	删去白棋，turn=2，moves.size()=1

读完自测

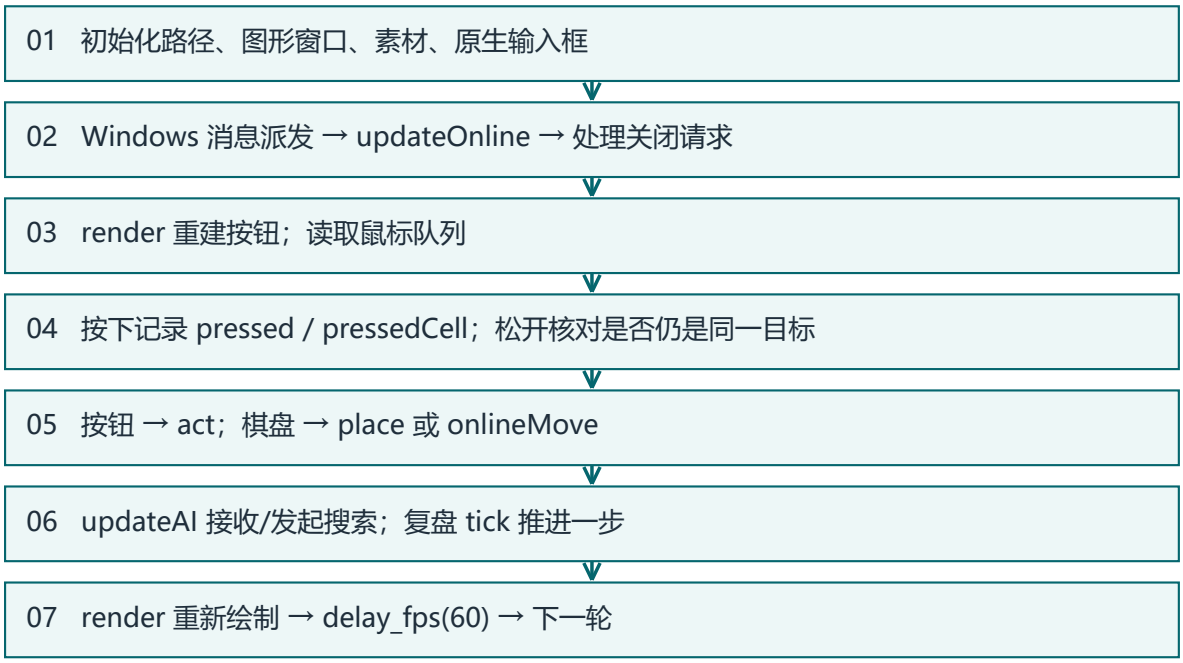
请遮住源码写出 place 的五步顺序，再解释为什么不能先把 turn 翻转后再存 Move。答案：那会把历史里的落子颜色记成对方。

CHAPTER 04

界面：一次点击怎样成为一手棋

打开 main.cpp；先读 595 行的入口骨架，再回头看它调用的函数。

图 4 主循环与输入主线



页面、弹窗、按钮是三套编号

类型	枚举值对应的职责
Page（37 行）	MAIN_MENU 主菜单；MODE_MENU 模式；GAME_PAGE 对局；SAVE_LIST 残局列表；REPLAY_LIST 复盘列表；REPLAY_PAGE 播放；DIFFICULTY_PAGE 难度；NETWORK_PAGE 联网流程。
Dialog（39 行）	NO_DIALOG 无；EXIT_SAVE 退出保存；RESTART_CONFIRM 重开；FINISHED_SAVE 终局保存；NAME_SAVE 命名；AGAIN 再战；DELETE_CONFIRM 删除；REPLAY_EXIT 退出复盘；CHOOSE_COLOR 选色。
Action（42 行）	A_START/CONTINUE/REPLAY/QUIT 菜单；A_LOCAL/AI 模式；A_EASY/MEDIUM/HARD 难度；A_BLACK/WHITE 颜色；A_BACK 返回；A_UNDO/EXIT_GAME/RESTART 对局；A_YES/NO/SAVE/CANCEL 弹窗；A_PLAY/SPEED1/SPEED2/PAUSE/RESUME/PREVIOUS/NEXT/REPLAY_EXIT 播放。
动态编号	列表加载=1000+2×下标；删除=1001+2×下标。联网按钮在 NetAction 中从 100 起，避开这些编号。

Button、SavedEntry 与全局变量

变量 / 成员	含义与取值
Button.id / x,y,w,h	动作编号；按钮左上角位置与宽高，单位像素。
Button.text / enabled / menu	按钮文字；能否命中；是否菜单风格（不贴普通底板）。
SavedEntry.name / path / mode	列表显示名、磁盘路径、模式字符串。
SavedEntry.valid / result	能否成功解析；结果数值，坏档用 -1。
BOARD_X / BOARD_Y	棋盘图片左上角 (100,200)，不是第一个交点。
FIRST_X / FIRST_Y / SPACING	首交点 (178,278)；相邻交点 66 像素。
MAIN_LABELS / MODE_LABELS	主菜单与模式菜单各 4 个宽字符串标签。
page / dialog / game	当前页面、当前本地弹窗、当前棋局。
replayPlayer / replayRecord / replayName	播放器、复盘完整元数据、复盘显示名。
aiWorker / gameMode	后台 AI 对象；LOCAL_TWO、HUMAN_AI、ONLINE 三种模式。
difficulty / humanStone	难度 0/1/2；人类执黑 1 或执白 2。
generation / deleteIndex	AI 请求版本号；待删除条目的下标（-1 表示没有）。
currentName / loadedPath	当前棋局名；载入残局路径，用于续局覆盖及完成后移除原残局。
buttons / entries	本帧可绘制的按钮；读盘产生的列表项目。
frames[90] / background / boardImage	菜单动画帧、页面背景、棋盘贴图的 EGE 图像指针。
blackImage / whiteImage / buttonImage	黑白棋子与按钮底板图像指针。
nameEdit / editFont	原生命名输入框句柄；它使用的字体句柄。
running / savingFinished	主循环继续标志；命名取消后该返回终局保存还是退出保存。

变量 / 成员	含义与取值
pressed / pressedCell / listOffset	按下的按钮编号、按下的格子编号、列表首项下标。
notice / noticeUntil	短提示内容及其到期毫秒时刻。
windowCloseRequested / originalWindowProc	跨回调传来的关闭请求；被替换前的窗口处理函数指针。

main（客户端）

main.cpp : 595

作用与接口 程序入口，返回 0 表示素材/控件加载成功，1 表示加载失败等早期错误。

执行顺序 把当前目录改为 exe 目录；设置 DPI；建 1800×1350 窗口；加载图片与 90 帧；建输入框并挂窗口回调；运行主循环；退出时停止网络与 AI，释放控件、字体、图片。

参数与局部量 path、n、slash 是 exe 路径、长度、末级分隔符；DpiFunction 是函数指针类型，dpi 是动态获取的 DPI 函数；loaded 是资源加载总结果；file 为动画文件名；started 为动画基准毫秒；message 为 Windows 消息；m 为 EGE 鼠标消息；hit 为命中按钮；cell 为松开时格子；images 为待释放图片数组；i 为循环下标。

对照源码时留意

主循环内可能多次 render，不能理解成每轮严格只画一次。delay_fps(60) 是限帧目标，不保证任何机器都恒定 60 帧。

gameWindowProc

main.cpp : 257

作用与接口 Win32 窗口回调，返回 LRESULT。

执行顺序 WM_CLOSE 只把请求标志原子置 1，稍后由主循环处理；其他消息交给原回调。

参数与局部量 hwnd 为窗口；msg 为消息种类；wp/lp 为消息附加参数。

对照源码时留意

本地页面点击窗口 X 会直接退出；想保存请走界面“退出”。联网对局的 X 则走退出/保存流程。

humanCanMove

main.cpp : 67

作用与接口 返回当前是否允许人类在棋盘上操作。

执行顺序 要求 GAME_PAGE、无本地弹窗、未终局；人机模式必须轮到人；联网模式还须 onlineCanMove。

boardCell

main.cpp : 360

作用与接口 把像素 x、y 转成 0..224 格子编号，失败返回 -1。

执行顺序 减去首交点；超出外缘 30 像素即拒绝；除以 66 得最近行列；重新算与交点偏移；圆形距离超过 30 像素拒绝；返回 row*15+col。

参数与局部量 dx/dy 先是到棋盘原点偏移，后是到最近交点偏移；col/row 为格子坐标。

对照源码时留意

例如 (640,740) 对应 (7,7)，编号 112；用 cell/15 还原行，用 cell%15 还原列。不能只按棋盘图片矩形判断落点。

addButton

main.cpp : 109

作用与接口 把按钮信息登记进本帧 buttons，不立即执行动作。

执行顺序 用参数组成局部 Button b，push_back 入数组。

参数与局部量 id 是动作；x/y/w/h 是矩形；label 是文字；enabled 默认 true；menu 默认 false。

contains

main.cpp : 116

作用与接口 判断像素 x、y 是否在按钮 b 的圆角形状内。

执行顺序 先检查外接矩形；把局部坐标夹在内部矩形上找最近点；比较距离平方与半径平方。

参数与局部量 r=21 圆角半径；lx/ly 为相对按钮左上角坐标；cx/cy 为夹取后的最近点。

hitButton

main.cpp : 123

作用与接口 返回鼠标下可用按钮 id，没命中返回 0。

执行顺序 i 从 0 到 buttons.size()-1；首个 enabled 且 contains 的按钮立即返回。

参数与局部量 x/y 是鼠标位置；i 为按钮下标。

对照源码时留意

源码注释写“从后往前”，实际是从前往后。发生重叠时先加入的可用按钮优先。

drawButtons

main.cpp : 129

作用与接口 按 buttons 绘制按钮外观，无返回值。

执行顺序 非菜单按钮先贴底板；若按下且仍位于按钮内就画灰色；设字体与禁用色；文字居中。

参数与局部量 mouseX/mouseY 是鼠标位置；i 是索引；b 是当前按钮的 const 引用。

toast

main.cpp : 94

作用与接口 设置两秒短提示。

执行顺序 将参数 text 存入 notice；noticeUntil=当前毫秒+2000；真正绘制在 render。

参数与局部量 text 是提示宽字符串，不是同名绘图函数。

font

main.cpp : 98

作用与接口 设置 EGE 文字字体与字号。

执行顺序 清零 LOGFONTW f；写负字号、抗锯齿选项和楷体；设透明文字背景。

参数与局部量 size 为像素高度；f 为字体配置对象。

text

main.cpp : 104

作用与接口 在 x、y 绘制 value，默认字号 32。

执行顺序 调用 font；设深棕色；outtextxy 输出宽字符串。

参数与局部量 value 为文字；size 为字号。

hideName

main.cpp : 145

作用与接口 隐藏命名框并把焦点交回游戏窗口。

执行顺序 ShowWindow 隐藏 nameEdit；SetFocus 切换焦点。

mainMenu

main.cpp : 147

作用与接口 切回主菜单并清理本地交互状态。

执行顺序 取消 AI；隐藏命名框；设 MAIN_MENU、NO_DIALOG、pressed=0。

newGame

main.cpp : 149

作用与接口 开始空棋局；keepName 默认 false。

执行顺序 取消旧 AI；隐藏输入框；game.reset；切 GAME_PAGE；按 keepName 决定是否清除名字与 loadedPath；人机模式弹选色。

参数与局部量 keepName=true 用于“重开同一局”，不是从头读取存档。

openList

main.cpp : 159

作用与接口 扫描残局或复盘，更新 entries 与列表页面。

执行顺序 按扩展名查文件；跳过目录/重解析点；readRecord 验证并取显示信息；按路径倒序排；夹紧滚动位置；非刷新且为空时只提示。

参数与局部量 isReplay 决定目录；refresh 是否保留滚动位置；dir 为目录；info 为查找结果；find 为搜索句柄；entry 为列表项；record 为解析记录；排序 lambda 的 a/b 为两个条目。

saveGame

main.cpp : 187

作用与接口 保存本地/人机棋局，成功返回 true。

执行顺序 去掉名称两端空白并检查长度；构造记录；残局可覆盖 loadedPath，完成局创建复盘；成功后更新 currentName；完成续局时尝试删除旧残局。

参数与局部量 name 按值传入，可在函数内修剪；first/last 是首尾有效字符下标；record 为待写记录。

对照源码时留意

联网保存走 onlineSave。删除原残局失败时会提示，但复盘本身已写成功。

savedSuccessfully

main.cpp : 215

作用与接口 保存完成后的页面收尾。

执行顺序 隐藏命名框；已结束则弹 AGAIN，未结束则回主菜单。

beginName

main.cpp : 219

作用与接口 准备保存：沿用旧名字或显示命名框。

执行顺序 记录 savingFinished；有 currentName 就直接保存，成功后收尾；否则设置 NAME_SAVE、清空并聚焦编辑框。

loadEntry

main.cpp : 229

作用与接口 加载列表中 index 对应文件。

执行顺序 先检查下标并重新读档；拒绝联网残局继续；取消 AI；复盘模式打开 replayPlayer，续局模式恢复 game 及模式、难度、颜色、名字、路径。

参数与局部量 record 为临时记录；isReplay 由 page 是否为 REPLAY_LIST 决定。

act

main.cpp : 262

作用与接口 按钮动作的总分发器，输入 id。

执行顺序 联网编号交给 onlineAction；>=1000 按偶奇区分载入和删除；其余 switch 处理菜单、对局、保存和复盘。

参数与局部量 index=(id-1000)/2 是列表下标；at 为最近人类落子下标；isReplay 为删除后刷新哪类列表；name[128] 为输入框读回缓冲。

对照源码时留意

重点跟读 A_UNDO、A_YES、A_NO：同一个“是/否”按钮，由 dialog 决定具体操作。悔棋先取消 AI，再撤掉玩家最近一手及其后的电脑应答。

act 中的路径	状态变化
A_EXIT_GAME → A_YES	未完弹 EXIT_SAVE；同意则 beginName；成功后回菜单。
落子致胜 → A_YES	FINISHED_SAVE → 命名/保存 → AGAIN；再同意则新局。
A_RESTART → A_YES	RESTART_CONFIRM → newGame(true)，清盘但保留名字。
A_REPLAY_EXIT → A_NO	先暂停，再取消退出询问；播放器仍暂停。
A_CANCEL	隐藏命名框，按 savingFinished 回到对应保存询问。

drawGame

main.cpp : 370

作用与接口 画对局或复盘；主要读取状态，登记对应按钮。

执行顺序 选择 shown 棋盘；铺背景与棋子；联网显示悬停预览和末手圈；终局画横幅与胜线；随后按复盘/联网/本地分支画侧栏。

参数与局部量 replay 是是否复盘；shown 是当前展示盘引用；title/status/banner 为标题文字；m 为正在画的一手；mx/my/cell 为悬停目标；last 为末手；endReason 为结束原因；dirs/d/sign/r/c/line 为胜线收集；step 为步数文字；free 为按钮启用条件；level 为难度文字；i 为遍历索引。

对照源码时留意

认输有胜负但没有连五，所以 END_RESIGN 跳过金圈胜线。绘制按 moves 遍历，不是逐格扫 cells；两份数据须保持一致。

render

main.cpp : 524

作用与接口 按 page 绘制整页，并重建可点击按钮。

执行顺序 先清 buttons；菜单按时间选动画帧；网络/棋局交给专门函数；列表只画最多 8 项；最后获取鼠标位置画按钮和未到期 toast。

参数与局部量 startedAt 为动画时间原点；frame=((当前-原点)%3000)*30/1000；ys/ids/levels 是菜单位置、动作、难度标签；id/i 为按钮编号与循环；at/entry/label 为当前列表项；name 为待删名；x/y 为鼠标位置；left/top 为提示框位置。

对照源码时留意

动画共 90 帧、每轮 3 秒；图形主循环目标 60 FPS，不代表素材动画也有 60 个不同帧/秒。

loadImage

main.cpp : 589

作用与接口 读取 path 图片，返回 PIMAGE，失败返回 NULL。

执行顺序 newimage 分配；getimage 加载；任一步失败就释放已分配 im。

参数与局部量 path 为窄字符串路径；im 为新图像指针。

这一章的检验

用自己的话复述：按下(7,7) → pressedCell=112 → 松开仍是112 → humanCanMove → place(7,7) → 下一帧画棋子。联网模式在哪一步不同？答：调用 onlineMove，只发请求，等服务端状态回来再显示实子。

CHAPTER 05

存档：保存历史，读回时重新验证

顺序：match_metadata.h → GameRecord → encodeRecord → parseRecord → 文件读写。

数据结构与结果约定

变量 / 成员	含义与取值
GameRecord.name / path	显示名（宽字符串）；实际文件路径，读写成功后补充，不写入文件正文。
GameRecord.mode	LOCAL_TWO、HUMAN_AI、ONLINE。
GameRecord.difficulty / human	AI 难度 0..2；人类或本机的颜色 1/2。
GameRecord.reason / version	结束原因枚举；读入格式版本。编码实际按 mode 选 2 或 4，不照抄 version。
GameRecord.started	开局 Unix 时间戳，单位秒；0 表示旧档未记录。
GameRecord.replay / game	是否复盘文件；棋盘与历史。联网未完局也只存复盘。
EndReason	END_UNKNOWN=0 未记录；END_FIVE=1 连五；END_DRAW=2 平局；END_RESIGN=3 认输；END_QUIT=4 主动退出；END_DISCONNECT=5 断线；END_RESTART=6 协商重开。

GameRecord::GameRecord

record_store.h : 21

作用与接口 创建默认单机记录。

执行顺序 mode=LOCAL_TWO，difficulty=0，human=1，reason 未知，version=4，started=0，replay=false；game 自动构造为空棋盘。

参数与局部量 默认 version=4 不代表本地写出来是 v4；看 encodeRecord。

validResult

match_metadata.h : 15

作用与接口 检查棋盘推导结果 boardResult 与记录 result、reason 是否相容。

执行顺序 先范围检查；认输要求盘面未结束且一方胜；连五和平局要求吻合；退出/断线/重开要求两结果均为 0；未知原因要求相等。

参数与局部量 boardResult 是按历史重放得到的结果；result 是文件/消息声称的结果；reason 是结束原因。

reasonText

match_metadata.h : 27

作用与接口 将 reason 转成中文宽字符串常量。

执行顺序 switch 映射六种已知原因；default 提示旧记录未记录。

matchTime

match_metadata.h : 40

作用与接口 把 Unix 秒 timestamp 格式化为本机当地时间。

执行顺序 0 返回旧档提示；转 time_t；localtime 拆解；失败返回无法显示；swprintf 生成日期时间。

参数与局部量 value 为 time_t；t 为 tm 指针；text[80] 为结果缓冲；tm_year 需加 1900，tm_mon 需加 1。

toUtf8

record_store.h : 25

作用与接口 wstring → UTF-8 string。

执行顺序 空串直接返回；第一次 WideCharToMultiByte 问所需字节数，第二次填入已分配 out。

参数与局部量 s 输入；n 目标字节数；out 目标缓冲。

fromUtf8

record_store.h : 35

作用与接口 UTF-8 string → wstring；无效 UTF-8 返回空串。

执行顺序 用 MB_ERR_INVALID_CHARS 严格测长度；n=0 失败；分配 out 后转换。

参数与局部量 s 输入字节；n 目标宽字符代码单元数；out 转换结果。

recordDirectories

record_store.h : 47

作用与接口 确保 data、unfinished、replays 三个目录存在。

执行顺序 创建失败但“已存在”可接受；再查询属性确认实际是目录。

参数与局部量 dirs 为三个路径；i 为索引；a 为属性位集合。

numberLine

record_store.h : 62

作用与接口 读取且校验一行“关键字 整数”；结果写入 value。

执行顺序 getline 读整行；istringstream 拆字段；必须 k==key，并且不存在 extra。

参数与局部量 input 输入流；key 期望关键字；value 输出整数引用；line 原行；row 行解析流；k 读到的键；extra 检测多余字段。

encodeRecord

record_store.h : 163

作用与接口 将记录 r 编成存档文本。

执行顺序 写文件头、名称、SIZE/MODE/难度/颜色；联网加原因与开始时间；写 RESULT/TURN/MOVES；逐手写 MOVE；最后 END。

参数与局部量 out 为字符串输出流；i 为手序下标；m 是这一手的 const 引用。

对照源码时留意

本地写 v2；联网写 v4。它只序列化，真正写盘前还要由 writeRecord 调 parseRecord 自检。

最小未完例子（只展示格式，不是整份源码）：

```
GOMOKU_LOCAL_SAVE 2
NAME 练习棋局
SIZE 15
MODE LOCAL_TWO
DIFFICULTY 0
HUMAN 1
RESULT 0
TURN 2
MOVES 1
MOVE 1 7 7 1
END
```

parseRecord

record_store.h : 77

作用与接口 解析 bytes；replay 指定预期文件类型；全部成功才赋值 output。

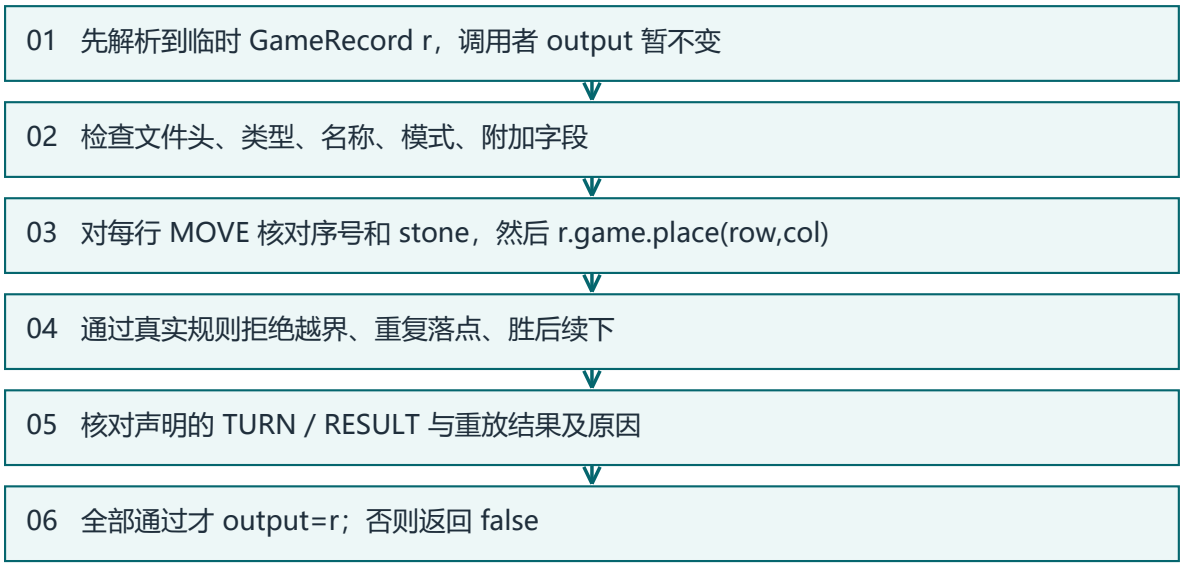
执行顺序 限制 2 MiB；移除回车；校验头与版本 1..4；读名称与模式；按版本读附加字段；逐步检查手序、棋色并调用 place；最后验证 END、结果、轮次、文件类型。

参数与局部量 normalized 是去回车文本；input 总流；r 临时记录；line/type/extra 当前行、关键字、多余字段；version 为头部版本；head/modeLine/timestamp/number/move 是不同段的解析流；size/result/turn/count 为声明的盘宽、结果、回合、手数；stamp 是时间文本；i/index/row/col/stone 是第 i 手与它的字段。

对照源码时留意

v4 只允许 ONLINE 复盘；v1 不能是 HUMAN_AI。只接受非零结果的本地复盘；v3/v4 ONLINE 允许 result=0。不应误读成所有版本都允许任意模式。

图 5 读档是一场有裁判的重放



readRecord

record_store.h : 146

作用与接口 从 path 读文件并解析，成功补 output.path。

执行顺序 CreateFileW 打开；检查长度；一次读取且要求实读长度相等；关闭句柄；交 parseRecord。

参数与局部量 replay 为期望类型；f 文件句柄；size 为 LARGE_INTEGER；data 为字节缓冲；got 为实读字节数；ok 汇总成功状态。

writeRecord

record_store.h : 189

作用与接口 把 r 安全写入磁盘；overwrite 为空表示创建新文件。

执行顺序 建目录；编码后反向解析自检；生成时间名和序号；CREATE_NEW 建同目录 .tmp；写完并刷新；MoveFileExW 改为正式名；失败清理临时文件。

参数与局部量 bytes 为编码文本；check 为自检对象；now 为当前本地时间；sequence 为 0..999 避让序号；filename 为文件名缓冲；path 为最终路径；temp 为临时路径；f 为写句柄；written 实写长度；ok 为状态。

对照源码时留意

新记录不会因显示名相同覆盖旧文件；只有显式 overwrite 才替换指定路径。先临时写再替换可避免直接把原档写到一半；仍可能因权限或 I/O 错误失败。

读完自测

把示例 MOVE 里的最后一个 1 改成 2 会怎样？parseRecord 在 stone!=r.game.turn 处失败。把 RESULT 改成 1 呢？重放没有连五，结果一致性校验失败。

CHAPTER 06

复盘：播放历史，不改原棋局

打开 replay_logic.h。重点区分“走到第几步”和“什么时候走下一步”。

变量 / 成员	含义与取值
ReplayPlayer.moves / board	完整历史副本；当前播放进度对应的展示棋盘。
cursor	已播放手数，0..moves.size()，不是最后一手下标。
started / playing	是否曾按播放；现在是否自动播放。暂停后 started 仍为 true。
interval / finalResult	每手间隔毫秒（3000 或 1500）；末尾才展示的最终结果。
deadline / remaining	下一手应播放的绝对毫秒时刻；暂停时距离下一手剩余毫秒。

ReplayPlayer::ReplayPlayer

replay_logic.h : 19

作用与接口 初始化暂停的空播放器。

执行顺序 cursor=0，started/playing=false，interval/remaining=3000，finalResult/deadline=0；board 自动初始化。

ReplayPlayer::open

replay_logic.h : 21

作用与接口 从 source 取历史，准备从空盘观看。

执行顺序 复制结果和历史；复位游标/播放标志/速度；清空展示盘；若历史为空，立刻显示 finalResult。

参数与局部量 source 是只读棋局引用。

对照源码时留意

零步认输也有终局结果，所以不能要求“至少落一子才显示结果”。

ReplayPlayer::seek

replay_logic.h : 29

作用与接口 跳到 position 手，超过末尾则夹到总手数。

执行顺序 清空 board，再重放前 cursor 手；到末尾才恢复 finalResult；remaining 设为 interval。

参数与局部量 position 是目标手数；i 为重放下标。

对照源码时留意

重放最多 225 手，简单可靠；不会删除 source 的历史。seek 自己不改 playing/deadline，上层只在暂停单步，tick 则会重新设 deadline。

ReplayPlayer::pause

replay_logic.h : 36

作用与接口 暂停并保留还需等待的时间。

执行顺序 若正在播，remaining=max(deadline-now,0)；playing=false。

参数与局部量 now 为调用者传入的当前毫秒。

ReplayPlayer::play

replay_logic.h : 41

作用与接口 从剩余倒计时开始/继续自动播放。

执行顺序 已到末尾就返回；否则 started/playing=true；deadline=now+remaining。

参数与局部量 now 为当前毫秒。

ReplayPlayer::speed

replay_logic.h : 46

作用与接口 改变一手间隔 milliseconds。

执行顺序 记住 active；pause；把 remaining 按新旧 interval 比例缩放；写新 interval；原先在播放就恢复。

参数与局部量 active 保存原 playing；now 当前毫秒。

对照源码时留意

调用者只传 3000 或 1500；函数本身没有检查零或负间隔。

ReplayPlayer::tick

replay_logic.h : 52

作用与接口 主循环每帧调用，按时自动前进一步。

执行顺序 仅在 playing 且 now>=deadline 且未到末尾时 seek(cursor+1)；下次 deadline=now+interval；到末尾停止。

参数与局部量 now 当前毫秒。

对照源码时留意

每次 tick 最多一手；卡顿后不会 while 循环补播所有错过的手。

事件 / 时间（毫秒）	内部变化
open；play(1000)	remaining=3000；deadline=4000
tick(3999) / tick(4000)	前者不动；后者播第 1 手，deadline=7000
pause(5000)	remaining=2000，playing=false
play(10000)	deadline=12000，暂停期间不计入等待
speed(1500,11000)	原来还剩 1000；换速后剩 500；deadline=11500

读完自测

为什么 finalResult 独立存一份？认输的胜者不能单靠棋盘连线算出；seek 到末尾时必须恢复原记录中的最终结果。

CHAPTER 07

AI：先找好点，再假设双方都认真下

打开 ai_engine.h。先读 choose，再读 candidates / pointScore，最后读 search。

AI 没有神秘的“理解棋盘”：它在副本上试落子，用棋型打分，再模拟对手回复。这里的候选截断和启发评分意味着它是有时间限制的近似搜索，不能把“困难”理解成必胜或穷尽全部着法。

变量 / 成员	含义与取值
AIPoint.row / col / score	候选落点行列；启发排序分。返回的 score 并不保证等于最终递归搜索值。
AIEntry.key / score	置换表局面键；该搜索结论的分数。
AIEntry.depth / flag / point	已搜索深度；0 精确值、1 下界、2 上界；最佳点编码 row*15+col，-1 无点。
AISearch.board / level / limit	棋盘副本；难度 0/1/2；普通候选上限：困难16，其他12。
cancel / deadline / interrupted	指向共享取消标志；搜索截止毫秒；此次搜索是否已超时/取消。
randomState	xorshift 伪随机状态，零种子会改成 1。
hash / zobrist[15][15][3]	当前盘面 64 位指纹；每格每色的随机键，颜色 0 不使用。
table	困难档启用的 65536 项置换表，key%size 选择槽位。
history[2][225]	颜色0/1对应黑/白，列为压平落点；累积造成剪枝的 depth*depth 奖励。
WIN	1000000000000LL，搜索胜分；比棋型启发分大很多。
completedDepth	最后完整完成的一轮根搜索深度；不是计划深度，也不是中断时最深递归层。

AIEntry::AIEntry

ai_engine.h : 18

作用与接口 初始化一个未使用的置换表槽。

执行顺序 key/score/flag=0，depth=-1，point=-1。

参数与局部量 depth=-1 使默认条目不能满足正常深度命中条件。

AISearch::AISearch

ai_engine.h : 203

作用与接口 为 source 创建一次独立搜索。

执行顺序 复制棋盘；按 difficulty 配置上限和时间；清 history；生成 Zobrist 随机数并合成初始 hash；困难档分配表。

参数与局部量 source 为原棋盘只读引用；difficulty 为难度；stop 是取消标志地址；seed 默认1234567；r/c/s 为格子与棋色循环。

对照源码时留意

困难预算2000毫秒，其他500毫秒；简单档不会进行迭代加深。初始化和所有子步骤也可能耗时，预算是协作式检查而非硬实时中断。

AISearch::stopped

ai_engine.h : 37

作用与接口 返回搜索是否应停止，并把 interrupted 锁定为 true。

执行顺序 原子读取 cancel；或比较当前毫秒与 deadline；任一成立就标记中断。

参数与局部量 InterlockedCompareExchange(cancel,0,0) 利用原子操作读取当前值。

对照源码时留意

volatile 本身不提供完整线程同步；这里依靠 Windows 原子操作及线程结束等待。cancel 由调用方保证指向有效变量。

AISearch::random

ai_engine.h : 44

作用与接口 返回下一次 32 位伪随机值。

执行顺序 randomState 依次执行左移13、右移17、左移5后的异或更新。

参数与局部量 没有局部变量；每次调用改变 randomState。

对照源码时留意

固定种子使算法顺序可重复；AIWorker 实际用时钟种子，所以每盘选择可能不同。不是密码学随机数。

AISearch::put

ai_engine.h : 50

作用与接口 仅在搜索副本格子上试放/撤回，同时维护 hash。

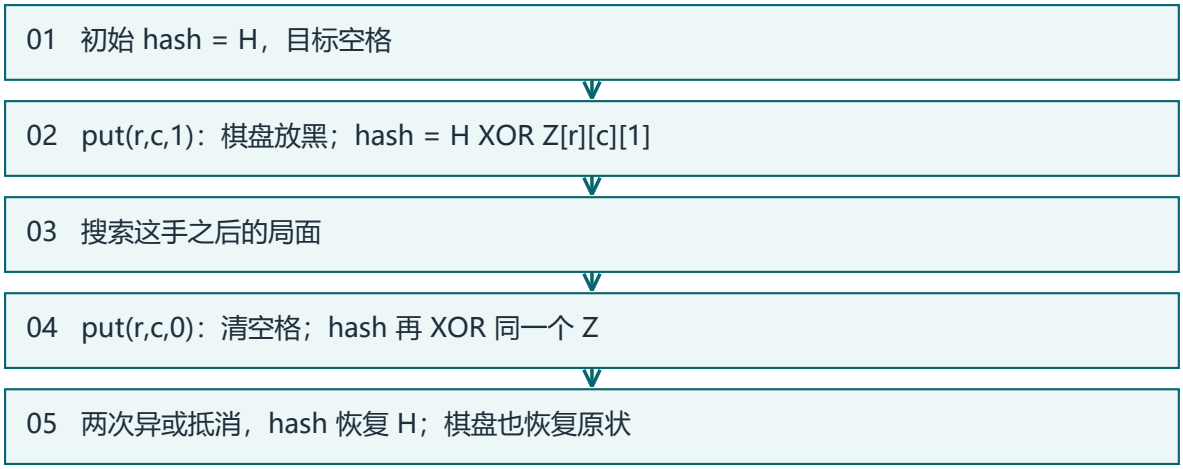
执行顺序 old 非零则异或掉旧键；写 board.cells；新 stone 非零则异或加入新键。

参数与局部量 r/c 目标位置；stone=0 表示撤子；old 是原来的颜色。

对照源码时留意

它不更新 turn、moves、result！递归通过参数 stone 管轮次，通过 wins 判试落子。不要把 put 当正式 place 使用。

图 6 可撤销的哈希更新



AISearch::pointScore

ai_engine.h : 61

作用与接口 估算在 r、c 放 stone 的局部棋型分。

执行顺序 每个方向取 -5..5 共11格；中心视为 X；同色X，空位点号，敌子/边界#；按从强到弱的模式只取该方向第一档命中分；四方向求和。

参数与局部量 dir 为方向；d 方向下标；sum 总分；line 11字符局部线；k 相对偏移；x/y 实际行列；value 本方向得分。

对照源码时留意

这是启发式字符串识别，不是严格棋型证明。find 在整段线中匹配，不强制命中的棋型包含中心，因此附近已有棋型也可能影响评分。

匹配形状（X 自己，. 空）	本方向 value
XXXXX	100000000；仍由 wins 决定正式胜负
.XXXX.	1000000
XXXX. / .XXXX / XXX.X / XX.XX / X.XXX	100000
.XXX. / .XX.X. / .X.XX.	10000
XXX. / .XXX / XX.X / X.XX	1000
.XX. / .X.X.	100
XX. / .XX	10；未匹配则0

AISearch::candidates

ai_engine.h : 94

作用与接口 为 stone 生成按优先级筛选的候选，preferred 默认 -1。

执行顺序 空盘只选中心；逐空格试自己能否赢、对方能否赢并恢复；普通点须两格邻域有子；算攻防分、历史奖励与优先点奖励；有直接赢点只返回 wins，否则有威胁只返回 blocks，否则排序截断 normal。

参数与局部量 wins/blocks/normal 为三类数组；occupied 是否非空盘；r/c 遍历格子；center 为中心点；win/block 是试子的胜判定；p 当前候选；nearby/x/y 是邻域扫描；排序 a/b 为候选引用。

对照源码时留意

“有多个必须挡点”不代表一手能全挡。wins/blocks 不走普通排序/数量截断，其 p.score 初始为0。中途 stopped 可能返回未完成 normal，choose 会按 interrupted 走兜底。

排序手算：某点自己得100，对手得1000，则普通启发分=100×10+1000×11=12000。防守权重略大，并不意味着电脑永远只防守；能直接赢的点在这一步之前就被优先选出。

AISearch::evaluate

ai_engine.h : 139

作用与接口 从 stone 的视角给当前整盘估分。

执行顺序 遍历所有有子的格；自己的 pointScore 加分，对方减分，得到代数和。

参数与局部量 value 为累计值；r/c 遍历位置；s 为格上实际棋色。

对照源码时留意

同一条棋型可能被多个棋子重复贡献；这是设计中的启发估计，不能解释成精确获胜概率。

AISearch::search

ai_engine.h : 153

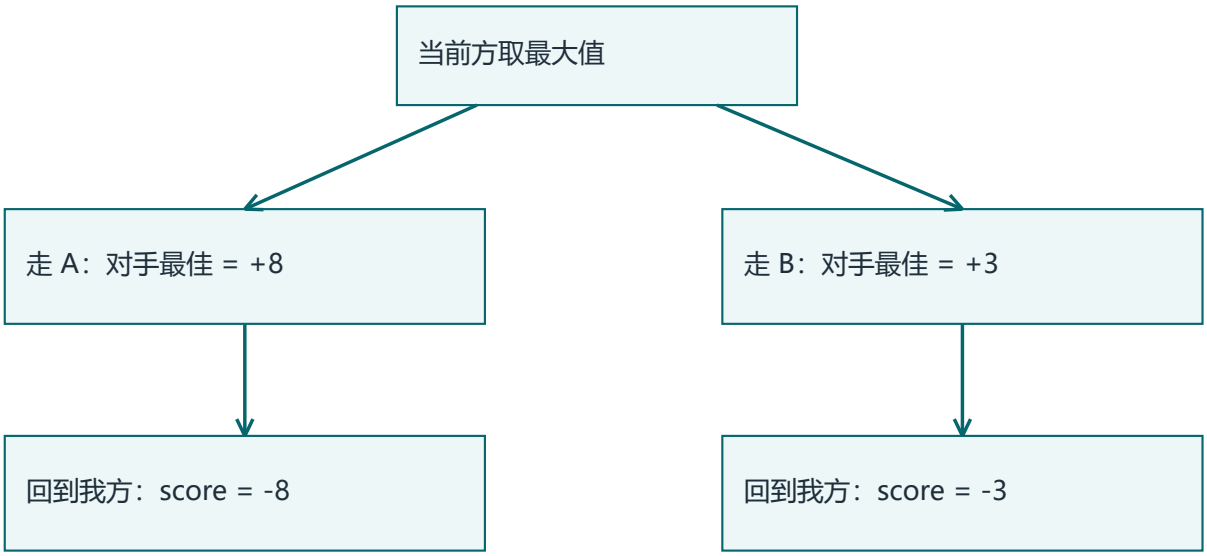
作用与接口 Negamax + Alpha-Beta：返回 stone 在剩余 depth 下的估计最佳分。

执行顺序 先处理停止和深度0；困难档查置换表；生成候选；逐个 put、判胜或递归取负；无论子搜索是否中断先撤子；更新 best/alpha；alpha>=beta 则剪枝；记录界限类型后返回。

参数与局部量 stone 是当前搜索方；depth 是剩余层数；alpha/beta 是搜索窗口；ply 是离根步数；oldAlpha/oldBeta 保存原窗口；key 合并盘面哈希和执棋方；entry 是表槽指针；preferred 是缓存点；points 候选表；best/bestPoint 当前最好分与点；i/p 当前候选；score 为换回本方视角后的子树分。

对照源码时留意

返回0可能表示中断、无候选或中性估分，必须结合 interrupted；上层不会把中断一半的轮次当完整答案。



示意分数：-3 > -8，所以选 B。每换一方就取负号；Alpha-Beta 的窗口也必须从 [alpha,beta] 变为 [-beta,-alpha]。

Alpha-Beta：什么时候可以不再看

alpha 表示当前已找到的最好下界；beta 是对手传来的可接受上界。当某候选 score 使 alpha>=beta，后面的同层候选已经不能改变上层决策，可直接 break。例如窗口 alpha=3、beta=5，某分支得7后，alpha变7，就不必再搜索该节点其余点。

递归表达式是 -search(3-stone, depth-1, -beta, -alpha, ply+1)。前面的负号是换视角；负号后的窗口顺序也反转。胜利返回 WIN-ply，偏好较早赢。根 choose 每个候选仍使用完整 [-WIN,WIN] 窗口，不应描述成根节点共享不断收窄的 alpha。

置换表：缓存也要说明“这是什么结论”

flag	读写的含义
0 精确值	本次 best 严格落在原窗口内部；命中且深度足够可直接使用。
1 下界	best>=oldBeta；只能保证至少这么好。命中后提高 alpha。
2 上界	best<=oldAlpha；只能保证不比这个更好。命中后降低 beta。

表槽由 key%65536 决定，槽位冲突会覆盖，但读取还要完整比较 entry->key==key。深度不够仍可借用最佳点 preferred 排序，不可直接借用分数。表和 history 在一次 AISearch 内复用；下一次 AI 落子会创建新的 AISearch，并非全局永续缓存。

AISearch::choose

ai_engine.h : 217

作用与接口 对外的选点入口，返回 AIPoint。

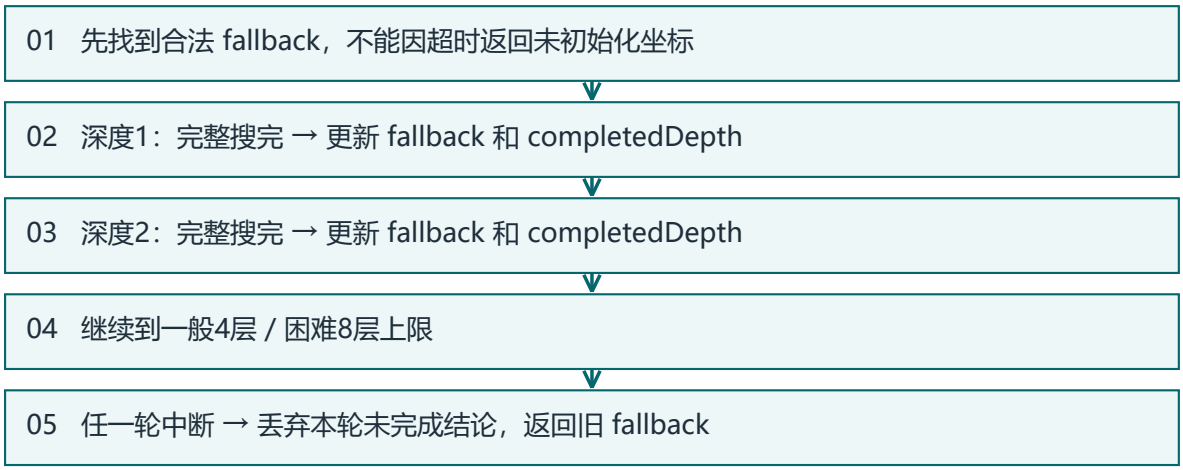
执行顺序 终局/满盘返回无效点；扫描首个空点兜底；生成根候选；直接赢或只有一个点就返回；简单档近优随机；一般/困难逐深度完整搜索，完成才替换 fallback，超时保留上一轮结果。

参数与局部量 fallback 保底/已完成最佳点；r/c 首空扫描；root 根候选；immediate 能否直接赢；count 简单档参与随机的前1..3个；depth 当前迭代深度；best 本轮最好分；next 本轮暂定最佳点；i/p 当前根点；value 根搜索分。

对照源码时留意

简单档仅在分数>0且至少为最高分90%的最多3点中随机；一般最多4层，困难最多8层，均不保证真的完成上限。returned score 是候选启发分，没有被赋为 value。

图 7 迭代加深与兜底



读完自测

为什么 search 递归返回后，先 put(...,0)，再检查 interrupted？答案：即便被取消，也必须把自己这一层试放的棋子撤回，否则父层会继续在脏棋盘和错误 hash 上工作。

CHAPTER 08

线程：让电脑思考时，窗口还能响应

AIWorker 负责运行搜索；main.cpp 的 updateAI 决定什么时候启动和采用结果。

变量 / 成员	含义与取值
AIWorker.thread	后台线程句柄；NULL 表示没有尚待回收的线程。
AIWorker.cancelled	跨线程停止标志，由原子操作读写。
AIWorker.snapshot	启动那一刻的棋盘副本，搜索不改界面 game。
AIWorker.difficulty / generation	此次任务难度；此次任务对应的棋局版本号。
AIWorker.answer	线程算出的 AIPoint；等待线程确实结束后才读取。

AIWorker::AIWorker

ai_engine.h : 275

作用与接口 建立尚未启动的工作器。

执行顺序 thread=NULL，cancelled=0，difficulty/generation=0；snapshot 自动初始化。

参数与局部量 answer 未显式初始化，但只在成功创建的线程结束并写出结果后由 take 使用。

AIWorker::run

ai_engine.h : 277

作用与接口 CreateThread 调用的静态入口，返回 DWORD 0。

执行顺序 把 context 转为 self；用快照、难度、取消地址、时钟种子建 AISearch search；choose 结果写入 answer。

参数与局部量 context 是传入的 this；self 为 AIWorker*；search 为本次搜索对象。

AIWorker::cancel

ai_engine.h : 282

作用与接口 发出协作式取消请求。

执行顺序 InterlockedExchange 把 cancelled 设1；不强制杀线程，也不立即关闭句柄。

AIWorker::start

ai_engine.h : 284

作用与接口 为 game 启动 level 难度、gen 版本的任务；返回创建是否成功。

执行顺序 已有 thread 就失败；复制 game；存参数；原子清取消标志；CreateThread(run,this)。

参数与局部量 game 是输入只读引用；level/gen 为任务参数。

AIWorker::take

ai_engine.h : 291

作用与接口 尝试取结果，经 point、gen 输出；未完成返回 false。

执行顺序 WaitForSingleObject(thread,0) 非阻塞检查；确认完成后复制 answer/generation；关句柄并置 NULL。

参数与局部量 point/gen 是输出引用。

对照源码时留意

线程已经完成也仍可能是被取消或旧棋局的结果；接受与否由 updateAI 的额外条件决定。

AIWorker::shutdown

ai_engine.h : 296

作用与接口 程序退出时彻底收尾。

执行顺序 cancel；若有线程则无限等待实际结束，再关句柄清NULL。

参数与局部量 等待期间不会继续该调用者的正常工作，适合退出清理。

cancelAI

main.cpp : 72

作用与接口 使旧 AI 任务失效并请求停止。

执行顺序 先 ++generation，再 aiWorker.cancel。

参数与局部量 全局 generation 是任务版本，不是棋盘步数。

lastHumanMove

main.cpp : 75

作用与接口 返回历史中玩家最近一手下标，找不到返回 -1。

执行顺序 从末尾倒序找 stone==humanStone。

参数与局部量 i 是有符号下标，避免空 vector 的 unsigned 下溢。

updateAI

main.cpp : 337

作用与接口 每帧收答案并按需启动新的 AI。

执行顺序 先 take；仅 gen==generation 且仍在无弹窗的人机未完局、轮到电脑时 place；再检查若轮到电脑且无线程则 start。

参数与局部量 point 为取得的落点；gen 为该答案携带的版本。

对照源码时留意

不是只检查“电脑线程算完了”。重开、读档、悔棋后 generation 已变，旧结果即使合法也应丢弃。

时刻	UI 与后台各自做什么
T0	generation=7；start 复制棋盘，任务记 gen=7。
T1	玩家按悔棋；cancelAI 把全局 generation 变8，取消旧任务。
T2	旧任务完成，take 得到 gen=7；7!=8，丢弃答案。
T3	若新棋局轮到电脑，启动 gen=8 的新任务。

不要让后台线程直接调用 game.place 或 EGE 绘图。现有边界是：后台只写自己的 answer；UI 在确认线程结束后接收结果，决定是否修改真实棋局。网络线程同样不直接画图，但使用队列加临界区交接消息。

CHAPTER 09

联网规则：两个人共享同一个裁判

打开 network_protocol.h。先不管 socket，先把它当成“输入命令，更新状态”的普通 C++ 模块。

联网棋局由服务端的 NetState 裁决。客户端只提交意图，服务端验证后向双方广播 STATE。这样能避免两边各算各的，出现黑方看见落子、白方却没有收到的分歧。网络延迟期间，客户端不会先把请求落成正式棋子。

变量 / 成员	含义与取值
NetState.game	服务端的权威棋局；客户端接收到的是它的快照。
round / phase	局号（每次 start 加1）；当前房间阶段，见下表。
owner / kind / question	请求发起者座位0/1；请求类型1重开、2退出、3悔棋、4认输；问答流水号。
reason / started	棋局结束原因 EndReason；开局 Unix 秒。
leaveReason / leaver / exitSerial	连接离开原因 LeaveReason；离开者座位；离开事件递增流水号。
saveStage[2]	两方保存进度：0尚未进入，1选择是否保存，2命名中，3本机处理完成。
present[2] / ready[2]	两方是否在房；是否已准备。
done[2] / again[2]	重开前是否已处理保存；终局后是否同意再战。
座位与颜色	座位0是房主，执黑1；座位1是朋友，执白2。颜色=座位+1；对方座位=1-本方座位。

NetPhase	准确含义
N_LOBBY=0	等待室；双方 READY 后开局。
N_PLAY=1	正常对局，可以提交 MOVE 或 ASK。
N_SELF=2	等待发起者自己确认操作。不是“等对方回答”。
N_PEER=3	重开/悔棋等待另一方同意。退出/认输不需要对方许可。
N_FINISH=4	棋局已分胜负或平局；双方独立保存并选择再战。
N_RESTART=5	双方已同意重开；双方处理保存后开始新局。
N_CLOSED=6	房间关闭；仍可处理本机保存和退出通知。

LeaveReason	连接层面的离开原因
LEAVE_NONE=0	没有离开事件
LEAVE_LOBBY=1 / LEAVE_HOST=2	朋友离开等待室 / 房主关闭等待室
LEAVE_QUIT=3	主动退出正在进行的棋局
LEAVE_DECLINE=4 / LEAVE_CANCEL=5	拒绝再战 / 已提交再战或重开后取消等待
LEAVE_DROP=6	连接意外中断

两个 reason 不能混用

EndReason 描述“棋局怎样结束”；LeaveReason 描述“连接为什么结束”。例如先认输、后拒绝再战：棋局 reason=END_RESIGN，连接 leaveReason=LEAVE_DECLINE。后续 socket 断开不应覆盖已经明确的原因。

NetState::NetState

network_protocol.h : 29

作用与接口 构造等待室状态。

执行顺序 round=0、phase=N_LOBBY；标量流水和原因复位；两方数组全false，saveStage=0；game 自动创建空盘。

NetState::start

network_protocol.h : 35

作用与接口 开始新一局。

执行顺序 局号加1；清棋盘；phase=N_PLAY；清请求类型及原因；记当前Unix秒；清准备、保存、再战标志。

参数与局部量 i 为双方座位数组下标。

对照源码时留意

present 保留；question、exitSerial 也不归零，流水号继续递增。owner=0 只表示默认值，不代表正在询问房主。

NetState::finish

network_protocol.h : 41

作用与接口 将当前局转为结束阶段。

执行顺序 reason=why；phase=N_FINISH；双方 saveStage=1。

参数与局部量 why 为 END_FIVE、END_DRAW 或 END_RESIGN。

对照源码时留意

它不计算胜者；连五由 place 设置 result，认输由 apply 设置 result。

NetState::lastMove

network_protocol.h : 43

作用与接口 寻找 player 座位最近一手下标，无则 -1。

执行顺序 从历史末尾找 stone==player+1。

参数与局部量 player 为0/1座位；i 为有符号历史下标。

NetState::closeRoom

network_protocol.h : 49

作用与接口 记录座位 p 因 why 发起离开。

执行顺序 写 leaver、leaveReason，增加 exitSerial；朋友在等待室离开时只清朋友在房/准备及房主准备，保留房间；其他场景设置未完结束原因并转 N_CLOSED。

参数与局部量 p 为离开者0/1；why 为 LeaveReason。

对照源码时留意

对已结束局或重开阶段保留原棋局 reason。该函数不总清 present，真正连接断开由 leave 清理。

NetState::leave

network_protocol.h : 58

作用与接口 处理底层连接消失。

执行顺序 若已不在房就返回；清 present/ready；尚未关闭的房间交 closeRoom(...,LEAVE_DROP)。

参数与局部量 p 为失联座位。

对照源码时留意

房间已关闭时不重写离开原因，防止“拒绝再战”被随后断线覆盖。

NetState::apply

network_protocol.h : 68

作用与接口 检查并执行座位 p 发来的 message；返回是否接受；event 输出可选提示事件。

执行顺序 检查座位/在房与命令格式；READY 单独处理；其余要求局号匹配；按命令验证阶段、手数、权限和问答编号，再改变状态。

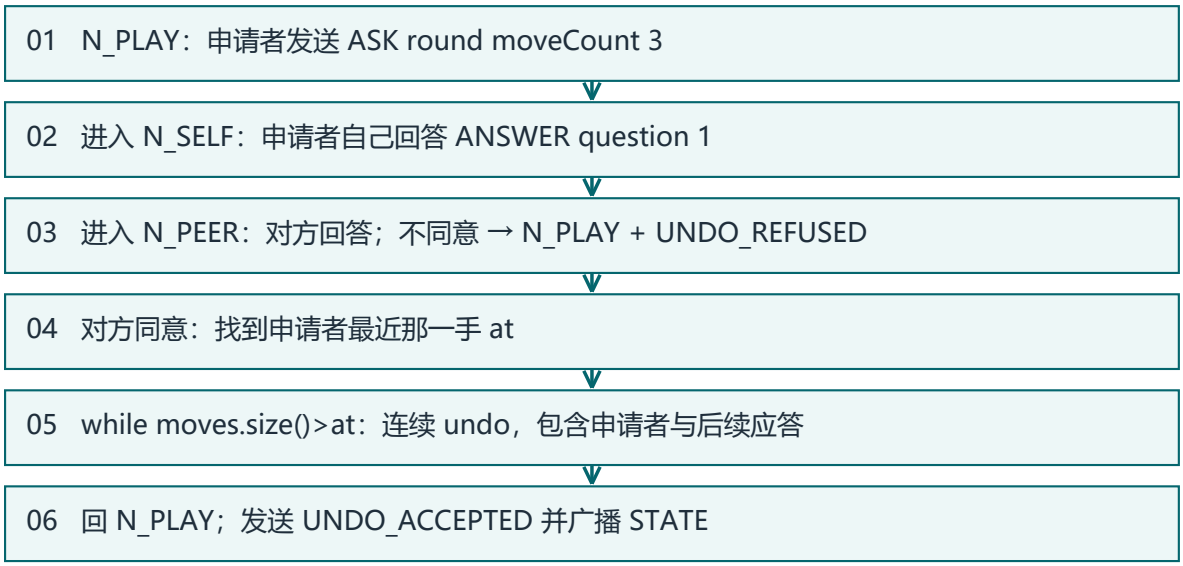
参数与局部量 in 为消息流；cmd 指令；extra 检测多余字段；r 局号；a/b/c 是按命令解释的整数；at 是悔棋目标历史下标。

对照源码时留意

源码若干注释把自己确认与对方确认写反。本书依据条件 p==owner / p!=owner 和分支执行顺序。
MOVE 不携带棋色，服务端根据座位和 game.turn 判断。

命令与 a/b/c 对应	验证与变化
READY a	a=0/1；只在等待室；双方在房且准备就 start。没有 round 参数。
MOVE r a b c	r=局号；a=当前已有手数；b=行；c=列。只允许 N_PLAY 且轮到 p+1；place 后可能 finish。
ASK r a b	a=已有手数；b=kind。悔棋还要求 lastMove(p)>=0；设置 owner、kind、question++，进入 N_SELF。
ANSWER r a b	a=question；b=0/1。N_SELF 只准 owner 回答；N_PEER 只准另一方回答。拒绝回 N_PLAY；kind2/4 自己确认即可退出/认输。
SAVE r a	a=1..3；只能结束、重开、关闭阶段，且未 done/again；仅同步保存进度。
DONE r	必须 N_RESTART 且自身 saveStage=3；两方 done 后 start。
AGAIN r	必须 N_FINISH 且自身 saveStage=3；两方 again 且都在房后 start。
CLOSE r a	a=LeaveReason；只接受与等待室/终局拒绝/已提交后取消相符的原因。

图 8 一次悔棋协商



手算：历史下标0黑、1白、2黑、3白，房主申请悔棋时 at=2。size=4>2 撤下标3，再 size=3>2 撤下标2，最终留下前两手。循环不是只撤到 size=at+1，因为申请者自己那手也要撤。

netEncode

network_protocol.h : 134

作用与接口 把 NetState s 编成一条 STATE 消息。

执行顺序 写局号/阶段/询问；写两方4类布尔标志；写手数和每手行列；最后写结果、原因、时间、保存进度和离开信息。

参数与局部量 o 为输出流；i 为座位或手序索引。

对照源码时留意

消息只传每手行列；颜色由黑白交替重放恢复。快照不是把结构体内存 memcpy 到网络上。

netDecode

network_protocol.h : 146

作用与接口 解析 message，成功后才整体赋给 result。

执行顺序 临时建 s；校验 STATE 头和范围；读双方标志和手数；逐手 place 重建；验证结果原因与其他范围、拒绝多余字段；写最终结果并输出。

参数与局部量 s 临时状态；tag 头标记；extra 多余字段；in 输入流；count 手数；outcome 消息宣称结果；i/r/c 是重放下标与行列。

对照源码时留意

这里做了多项格式和规则校验，但不是对所有字段组合做完整状态机证明。不要把它描述为能够识别一切恶意状态；权威性主要来自服务器执行 apply。

CHAPTER 10

TCP：收到的是字节，不是一条条消息

打开 network_transport.h。把“如何运字节”和“这一手能不能下”分开理解。

TCP 保证的是有序字节流。一次 send 不保证对应一次 recv：一条消息可能分几次到达，多条也可能一起到达。本工程给每条正文前加4字节网络序长度，接收方根据长度从累积缓冲区拆出完整消息。

变量 / 成员	含义与取值
NetSocket.fd	原始 SOCKET；INVALID_SOCKET 表示没有连接。
input / output	尚待拆帧的接收字节；尚未完全发送的字节。不是消息队列。
received / sent	最近成功收/发的毫秒时刻。received 用于15秒超时；sent 被记录但当前没有其他逻辑读取。
failed	queue 遇非法长度或缓冲超限时设true，pump 随后失败。
NetClient.mutex / thread / stopFlag	保护消息队列的临界区；后台线程句柄；原子停止标志。
incoming / outgoing	收到后等待UI取走 / UI提交后等待网络线程发送的完整字符串队列。
address / hello / port	目标IPv4文本、建立连接后发出的握手正文、目标端口。

NetSocket::NetSocket

network_transport.h : 24

作用与接口 构造尚未连接的流缓冲对象。

执行顺序 fd=INVALID_SOCKET；received=当前毫秒；sent=0；failed=false；字符串自动为空。

NetSocket::close

network_transport.h : 26

作用与接口 关闭套接字并清空收发字节。

执行顺序 若 fd 有效调用 closesocket；设无效；清 input/output。

参数与局部量 它不自动重置 failed/received，下一次 attach 会重置必要状态。

NetSocket::attach

network_transport.h : 29

作用与接口 接管 SOCKET s 并设为非阻塞。

执行顺序 先关闭旧连接；清 failed；保存 s；ioctlsocket(FIONBIO)；重设 received。

参数与局部量 s 是接管的句柄；mode=1 代表非阻塞。

对照源码时留意

后续由此对象负责 close。当前代码没有检查 ioctlsocket 的返回值。

NetSocket::queue

network_transport.h : 31

作用与接口 把一条正文 message 封装到待发送字节缓冲，返回是否成功。

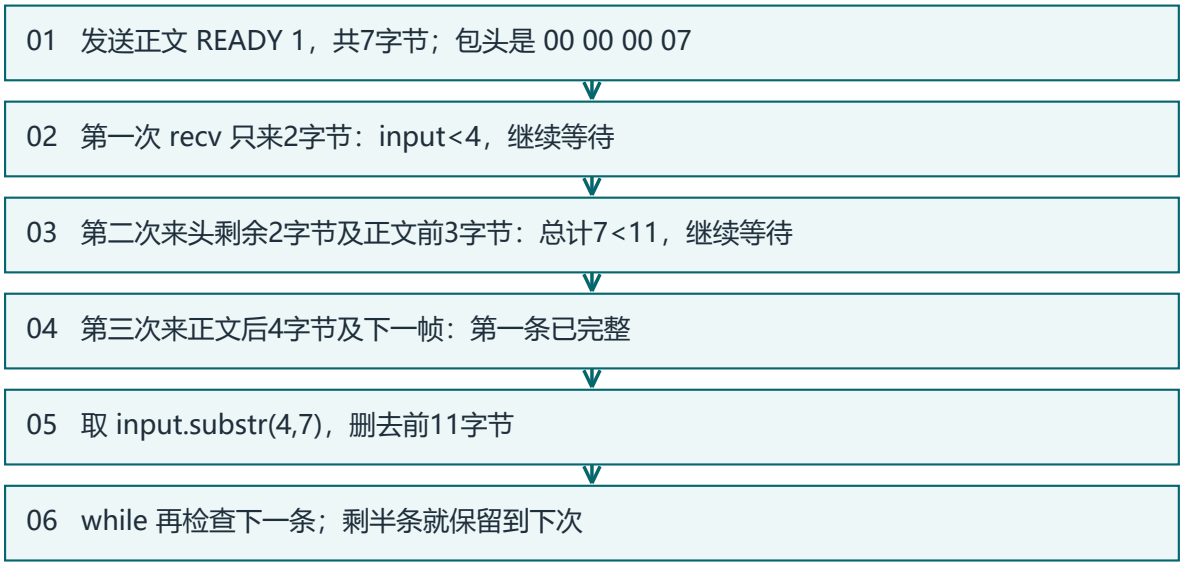
执行顺序 拒绝空正文、>8192字节或总缓冲>65536；用 htonl 得4字节长度；追加长度头与正文。

参数与局部量 n 为网络序正文长度。

对照源码时留意

queue 只入缓冲，不表示已经送达。Windows 中 unsigned long 为4字节，这段实现依赖当前平台。

图 9 长度帧的接收例子



NetSocket::pump

network_transport.h : 43

作用与接口 尝试发送并接收；把完整正文追加到 messages；false 表示连接不可继续。

执行顺序 先发送 output 的一部分并移除实发字节；最多16次 recv；把新字节追加 input；while 拆完整帧；遇零长度、超限、断开或超时失败；暂时无数据 WSAEWOULDBLOCK 不算故障。

参数与局部量 messages 为输出数组引用；n 分别为 send/recv 实际字节数；bytes[4096] 为接收块；iteration 限制本轮读取次数；length 为从包头取出的正文长度。

对照源码时留意

send 可能只发一部分。recv 返回0是有序关闭；<0才检查错误码。messages 超256也失败；这些界限防止缓冲无限增长。

NetClient::NetClient

network_transport.h : 162

作用与接口 构造客户端通信对象。

执行顺序 thread=NULL，stopFlag=0，port=0；InitializeCriticalSection 建锁。

NetClient::~NetClient

network_transport.h : 164

作用与接口 对象销毁时释放线程和锁。

执行顺序 先 stop 等线程退出，再 DeleteCriticalSection。

参数与局部量 析构函数名以 ~ 开头，随对象生命周期自动调用。

NetClient::push

network_transport.h : 95

作用与接口 网络线程把字符串 s 放入 incoming。

执行顺序 进锁；未满256则追加；否则置 stopFlag；离锁。

参数与局部量 s 是交给UI的完整消息或内部通知。

NetClient::entry

network_transport.h : 97

作用与接口 Win32线程静态入口。

执行顺序 将 LPVOID p 转回 NetClient*，调用成员 run，再返回0。

参数与局部量 p 是 CreateThread 传来的 this。

NetClient::run

network_transport.h : 99

作用与接口 后台线程的连接与收发主循环。

执行顺序 初始化Winsock；创建非阻塞TCP；connect并用select等最多约8秒；连上后queue hello；每轮取走outgoing、发心跳、pump、处理PING/PONG与延迟，其他消息push给UI；失败push LOST；退出清理。

参数与局部量 data 为Winsock信息；sock 为流封装；fd 为原句柄；target 为IPv4地址结构；rc/error 为connect结果和错误码；connected 是否完成；start 为等待开始毫秒；writeSet/errorSet 为select集合；wait 是10毫秒超时；n 为select结果；e/len 为SO_ERROR值及长度；ping 为最近本机心跳时间；pending 为本轮出站消息副本；ok/alive 为入缓冲及连接结果；request 为PING输出流；messages 为收到的正文；i 为索引；stamp/value 为PONG时间解析流与值；latency 为内部LATENCY通知流。

对照源码时留意

ping 时间由本机生成并由服务器原样回传，因此测的是往返时间，不需要两台机器时钟一致。线程每轮 Sleep(10)，没有进行图形绘制。

NetClient::stop

network_transport.h : 167

作用与接口 停止并回收后台线程，再清队列。

执行顺序 原子置 stopFlag；有线程则等待结束、关句柄、清NULL；在锁保护下清 incoming/outgoing。

NetClient::start

network_transport.h : 173

作用与接口 用 ip、p、greeting 启动连接任务，返回建线程是否成功。

执行顺序 先 stop 旧任务；保存地址、端口、握手；清停止标志；CreateThread(entry,this)。

参数与局部量 ip 为点分十进制IPv4；p 是端口，不是座位；greeting 为HELLO正文。

对照源码时留意

true 只表示线程建成，不表示已连接成功；连接成功需UI收到 WELCOME。

NetClient::sendMessage

network_transport.h : 179

作用与接口 UI提交一条出站 message。

执行顺序 在临界区内，outgoing 未满256就追加。

参数与局部量 当前实现队列满时静默丢弃，没有返回失败；不要理解成每次调用都确保消息入队。

NetClient::take

network_transport.h : 181

作用与接口 UI非阻塞取一条 incoming，经 message 输出。

执行顺序 进锁检查 found；有消息取front并pop_front；离锁，返回 found。

参数与局部量 found 为队列非空判断。

对照源码时留意

此处“不阻塞等待消息”不等于完全不等待：争用临界区时仍可能短暂等待锁。

读完自测

为什么 pending.swap(outgoing) 放在锁内，而逐条 sock.queue 放在锁外？答案：只在交接队列时持锁，实际处理在网络线程自己的局部队列上完成，减小 UI 被锁住的时间。

CHAPTER 11

服务端：一个循环，管理两个人的房间

打开 server_main.cpp。这里没有 EGE 图形界面，也不使用客户端的全局 game。

局部结构 Peer 与全部入口变量

变量 / 成员	含义与取值
Peer.socket / player	这一条连接的 NetSocket；座位0/1，-1表示尚未握手。
Peer.reject / accepted	是否已拒绝（等待错误消息发完再关闭）；接受连接的毫秒时刻。
peers[8]	8个TCP连接槽位，用于处理未握手/拒绝中的连接；不是8人棋局。
argc / argv	进程参数数量及文本数组；要求4项：exe、端口、房主口令、父进程PID。
port / token / parent	监听端口；房主握手口令；父进程PID。
parentProcess / data	用于观察父进程是否退出的句柄；Winsock初始化信息。
listener / exclusive / address	监听socket；独占端口开关；绑定所有本机IPv4网卡的地址结构。
eventName / readyEvent / mode	与客户端约定的就绪事件名和句柄；非阻塞模式值1。
state / hadHost	权威 NetState；是否曾有房主成功入房。
created / closedAt / ping	服务端创建时刻；关房开始时刻；上次发心跳时刻（均毫秒）。
watchedExit / ackSlot	已处理的离开流水；待发送 CLOSE_ACK 的连接槽下标，-1无。
seen[2] / ackAt	两座位是否确认当前离开通知；开始等确认的时刻。
run / incoming / slot	循环开关；新accept的socket；可用连接槽下标。
changed / p / messages / alive	是否需广播状态；当前Peer引用；收到正文数组；连接是否仍有效。
i / j / k / msg	连接槽索引、消息索引、广播扫描索引；当前消息的const引用。
response / serial / extra	SEEN解析流、退出流水号、检测多余字段的字符串。

变量 / 成员	含义与取值
input / cmd / version / role / key	HELLO解析流、指令、协议版本文本、HOST/GUEST身份、收到的口令。
host / error / event	是否房主身份；拒绝原因字符串；apply产生的提示事件。

Peer::Peer

server_main.cpp : 55

作用与接口 初始化尚未握手的连接槽。

执行顺序 player=-1，reject=false，accepted=0；内含 socket 自行构造。

参数与局部量 它是定义在 main 函数内部的局部结构体，仅在该函数作用域可用。

main（服务端）

server_main.cpp : 17

作用与接口 独立服务进程入口；管理监听、握手、权威状态和广播。

执行顺序 验证参数；OpenProcess观察父进程；初始化Winsock并绑定监听；通知就绪事件；循环accept、pump、握手或apply、检测断线、确认离开、广播与心跳；最后关闭全部资源。

参数与局部量 所有局部变量见上表。返回2表示参数错误，3表示Winsock初始化失败，4表示绑定/监听失败，0为循环正常结束。

对照源码时留意

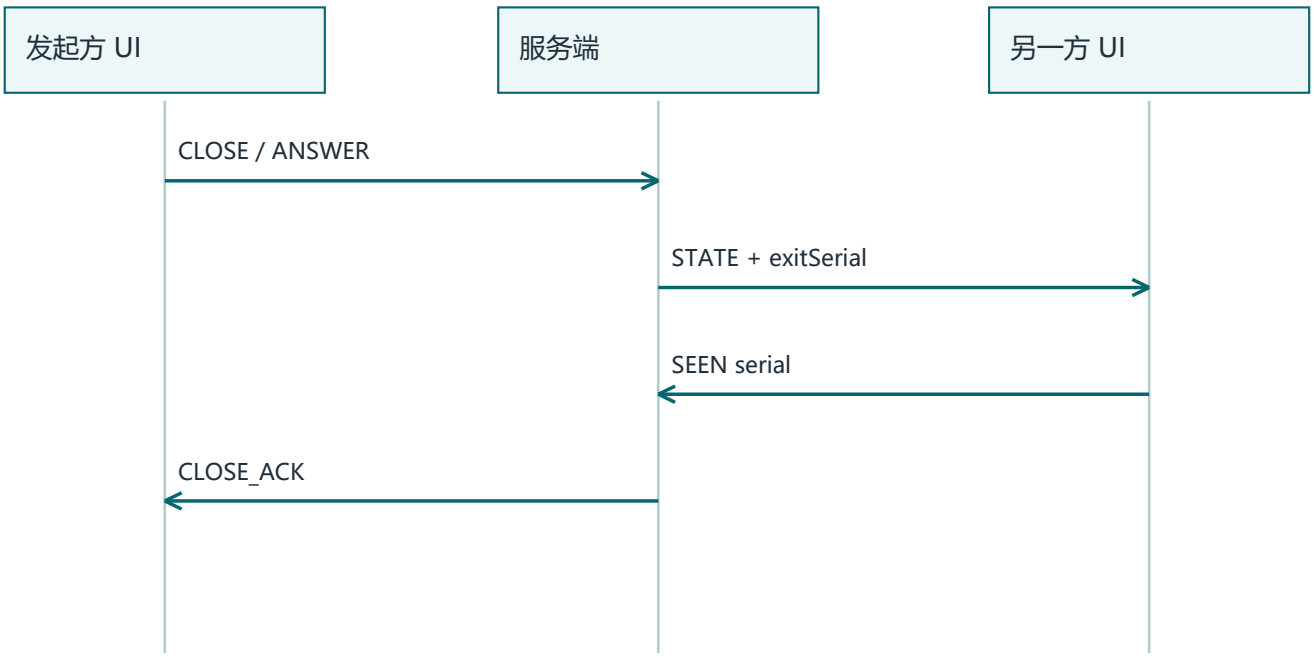
这个 main 与客户端 main 属于两个不同exe。房主客户端退出后服务端观察父句柄退出，避免留下后台房间。

图 10 服务端一轮做什么



握手检查的先后顺序

检查	失败消息 / 意义
HELLO + 版本2 + 合法身份 + 无多余字段	ERROR VERSION；版本与格式共同检查。
房主口令 key==token	ERROR AUTH；朋友无需房主口令。
phase==N_LOBBY	ERROR STARTED；对局中不允许新加入，不支持中断重连。
对应座位尚未占用	ERROR FULL；房主和朋友各一个座位。
朋友加入前房主已在	ERROR WAIT_HOST。
通过	设置player/present；房主记hadHost；发送WELCOME 0/1，触发STATE广播。



服务端等对方确认收到离开状态，或等待达到 3 秒，才允许发起方完成关闭。超时确认不等于对方必然收到。

为什么拒绝后不立刻 closesocket？Peer.reject 先置真，等 output 变空再关闭，使 ERROR 有机会发送出去。为什么每次非法命令后还发 STATE？客户端可能拿着过期状态操作，服务端用 EVENT INVALID 和当前快照让它重新对齐。

读完自测

peers[8] 中暂时有3条连接，是否意味可三人对战？不是。槽位是运输层的连接管理；合法身份最终只映射到 present[0] 与 present[1] 两个座位。

CHAPTER 12

联网界面：把状态变成可操作的页面

最后读 online_ui.h。它依赖 main.cpp 已声明的全局量和绘图函数，所以被 include 在 main.cpp 的第249行。

这个头文件不是可独立编译的“纯组件”。它直接使用 page、game、nameEdit、toast、mainMenu 等客户端全局符号；包含位置是依赖关系的一部分。把它随意移到文件最上面，可能出现名字尚未声明。

先把三种 stage 分开

状态变量	各值的含义
netStage（联网连接页面）	0选择创建/加入；1创建表单；2加入表单；3连接中；4等待室；5开局前关闭提示。
archiveStage（本机保存UI）	0正常对局；1询问保存；2命名；3选择再战；4已提交等待对方；5结束后返回。
netState.phase（权威房间）	N_LOBBY / N_PLAY / N_SELF / N_PEER / N_FINISH / N_RESTART / N_CLOSED；决定操作是否允许。

变量 / 成员	含义与取值
netClient / netState	后台网络对象；最近一次服务端权威快照。
serverProcess / serverReady	房主启动的服务进程句柄；等待监听成功的事件句柄。
ipEdit / portEdit	IP与端口的原生编辑框句柄，由客户端main创建。
netRole / archiveRound	本机座位：-1未分配、0房主、1朋友；已触发保存流程的局号，-1未触发。
netConnected / netPending	是否收到WELCOME并处于连接状态；是否在等操作后的状态更新。
netHosting / netStarted / netLost	此次是否创建房间；是否已经开局；是否已失联或房间正常关闭后禁止继续操作。
lobbyExit / closeAfterOnline	等待室退出询问标志；联网收尾后还要关闭整个程序。
netLeaving / netExitAck / netMovePending	正在离开并等待确认；已收到CLOSE_ACK；待确认的操作是否落子。

变量 / 成员	含义与取值
netSeenExit / netLatency	本机已处理的离开流水；往返延迟毫秒，-1尚未测到。
netAutoHomeAt / netLeaveDeadline	自动返回的毫秒时刻（0无）；等待离开确认的截止时刻。
netPendingAt / netConnectAt / netDeadline	最近操作发送时刻；计划启动连接时刻（启动后清0）；连接阶段总截止时刻。
netEndNotice / copyAddress	结束提示；复制连接信息时使用的单个IP文本。
netAddress / netGreeting / netPort	实际连接IPv4、HELLO正文、端口（默认5000）。
netError / hostAddresses	连接页错误文字；本机候选地址显示文字。

NetAction 枚举	调用意图
A_ONLINE / A_NET_HOST / A_NET_JOIN	进入联网、创建、加入。
A_NET_BACK / CONNECT / CANCEL	返回、连接、取消连接（后两者均带 A_NET_ 前缀）。
A_NET_READY / LEAVE / COPY	准备切换、等待室退出询问、复制地址端口。
A_NET_YES / NO	本机/对方确认弹窗的答复，或等待室退出确认。
A_NET_RESTART / EXIT / UNDO / RESIGN	分别对应 ASK kind=1/2/3/4。
A_NET_SAVE_YES / SAVE_NO / WRITE / NAME_CANCEL	进入命名、跳过保存、写入本机、取消命名。
A_NET_AGAIN / HOME	同意再战、请求结束连接并返回。

onlineForms

online_ui.h : 46

作用与接口 按 visible 控制联网表单输入框。

执行顺序 IP仅在 netStage=2 时显示；端口在 visible 时显示；焦点放到适合的框。

参数与局部量 visible 为显示开关。

onlineStop

online_ui.h : 54

作用与接口 停止连接并回收本机启动的服务进程。

执行顺序 停 netClient；清连接与待确认；服务进程等50毫秒未退就终止；关进程/事件句柄；隐藏表单。

参数与局部量 不负责完整重置所有联网字段，完整重置由 onlineEnter 处理。

onlineHome

online_ui.h : 68

作用与接口 完成联网清理后回主菜单。

执行顺序 onlineStop、hideName；复位保存阶段/开局标志/游戏模式；mainMenu；若 closeAfterOnline 则停止程序。

onlineEnter

online_ui.h : 75

作用与接口 从零进入联网菜单。

执行顺序 停止旧连接、取消AI、隐藏命名；重新构造 NetState；重置身份、连接页面、保存流程、离开标志、提示与延迟；page=NETWORK_PAGE。

localAddresses

online_ui.h : 85

作用与接口 枚举本机活动IPv4地址，返回显示文本，并设置 copyAddress。

执行顺序 GetAdaptersAddresses，缓冲不足则扩容重试；遍历启用网卡和单播地址；排除回环；优先收集100.64..100.127段；否则取首个普通地址。

参数与局部量 size/bytes 为缓冲长度和字节数组；list 为链表头；rc 为API结果；tailscale/other 为候选文本；a 是网卡节点；u 是地址节点；b 为IPv4四字节指针；value 为格式化地址缓冲。

对照源码时留意

这里按地址段启发式标记“Tailscale”，没有验证网卡确实属于Tailscale。界面也提示人工核对；识别成功不等于网络已经连通。

onlineCanMove

online_ui.h : 120

作用与接口 联网规则下是否允许本机点棋盘。

执行顺序 要求已连接、未丢失、未离开、无待确认、非保存阶段、N_PLAY、座位有效，且turn==netRole+1。

onlineSend

online_ui.h : 127

作用与接口 提交需要等待状态更新的命令 cmd。

执行顺序 若仍连接且未lost，sendMessage；设netPending；记录netPendingAt。

参数与局部量 cmd 为完整正文。

对照源码时留意

netPending 只是UI防连点标志，不是TCP送达证明。

onlineCommand

online_ui.h : 132

作用与接口 生成“cmd 当前局号”前缀字符串。

执行顺序 ostringstream out 写命令与 netState.round，再返回str。

参数与局部量 cmd 为命令名；out 为输出流。

onlineMove

online_ui.h : 136

作用与接口 把压平格子编号 cell 变成 MOVE 请求。

执行顺序 检查可下与空格；设netMovePending；拼命令、已有手数、cell/15、cell%15；onlineSend。

参数与局部量 out 为命令流；cell 来自 boardCell 的合法结果。

对照源码时留意

函数本身不检查cell越界，依赖调用者。不会在这里 game.place。

onlineAsk

online_ui.h : 143

作用与接口 提出 kind 类型的操作询问。

执行顺序 要求已连接、无待确认、N_PLAY；拼ASK、局号、手数、kind；发送。

参数与局部量 kind=1重开、2退出、3悔棋、4认输；out 为消息流。

onlineAnswer

online_ui.h : 149

作用与接口 针对当前 question 答复 yes。

执行顺序 拼ANSWER、局号、问答编号、0/1；onlineSend。

参数与局部量 yes 为是否同意；out 为消息流。

onlineSaveStatus

online_ui.h : 155

作用与接口 报告本机保存进度 stage。

执行顺序 若连接未lost，直接 sendMessage 发送SAVE，不设置netPending。

参数与局部量 stage=1/2/3；o 为输出流。

对照源码时留意

这是状态通知；为了不锁死本地命名流程，没有走 onlineSend。

onlineDeparture

online_ui.h : 159

作用与接口 提交离开原因 why，并开始等待退出确认。

执行顺序 已netLeaving就返回；置标志；设6秒deadline并取消自动返回；发送CLOSE。

参数与局部量 why 为LeaveReason；o 为命令流。

onlineRequestHome

online_ui.h : 167

作用与接口 根据当前阶段选择正确退出方式。

执行顺序 已离开则不重复；丢失/已关房直接onlineHome；未开局走等待室离开；终局按是否已提交再战选取消或拒绝；重开等待选取消。

参数与局部量 它不处理正在正常下棋时直接退出；那条路径先 onlineAsk(2)。

onlineArchiveDone

online_ui.h : 178

作用与接口 本机保存选择完成后的分流。

执行顺序 隐藏命名；报告SAVE 3；若连接结束，进入阶段5并等ACK或2秒返回；重开则阶段4并发送DONE；普通终局则阶段3让用户选再战；需关程序则继续请求退出。

参数与局部量 没有另建棋谱；它只推进保存后的交互。

onlineSave

online_ui.h : 200

作用与接口 以 raw 名称把权威历史存为本机联网复盘。

执行顺序 修剪/校验名称；枚举已有合法复盘名称；重名附加(2)、(3)...且截断基名控制长度；组装ONLINE记录及本机颜色/结果原因/开局时间；writeRecord。

参数与局部量 first/last 为首尾位置；name 清理后的名称；names 已有名数组；info/find 为文件枚举；r 为读回记录；base 为原名；i 为重名序号；suffix 为编号缓冲；record 为待写记录。

对照源码时留意

联网未完也 replay=true；只保存已收到确认的 game，不记录尚未获确认的落子请求。

onlineName

online_ui.h : 230

作用与接口 打开联网命名框并生成默认名字。

执行顺序 archiveStage=2；报告SAVE 2；取本地时间组成“联网年月日_时分秒”；显示并聚焦nameEdit。

参数与局部量 t 为SYSTEMTIME；value 为名称缓冲。

onlineConnect

online_ui.h : 236

作用与接口 验证表单，必要时启动服务端，并进入异步连接阶段。

执行顺序 端口纯数字且1..65535；选择本地回环/填写IP并校验IPv4；记住有效表单；朋友准备HELLO 2 GUEST；房主生成GUID口令、建就绪事件、隐藏启动服务器；隐藏表单并设10秒总deadline。

参数与局部量 portText/ipText 是输入缓冲；p 是端口字符串；number 为解析整数；a/b/c/d 为IPv4四段，extra 检测尾随字符；id 为GUID；token 为口令文本；eventName 为事件名；directory/exe 为目录与服务exe路径；line 为宽输出流；args 为可修改命令行；si 为启动信息；pi 为进程/线程句柄结果。

对照源码时留意

CreateProcess 参数包含端口、口令和父PID。这里没有立即连接：updateOnline 等serverReady亮起，避免服务端还没listen就连接失败。

onlineLobbyLeave

online_ui.h : 293

作用与接口 打开等待室离开询问。

执行顺序 lobbyExit=true；若本机已准备，先发送READY 0取消准备。

参数与局部量 此时还没真正CLOSE；用户随后可选择否。

onlineAction

online_ui.h : 299

作用与接口 处理所有 NetAction 按钮。

执行顺序 创建/加入设置表单并恢复INI；连接/取消转相应函数；准备/询问/保存/再战按按钮分流；复制按钮组装IP端口并写系统剪贴板。

参数与局部量 id 动作；value 在不同分支分别是INI/命名缓冲或复制输出流；content 为复制正文；ok 成功标志；memory 为可移动全局内存；pointer 为GlobalLock后地址。

对照源码时留意

剪贴板 SetClipboardData 成功后内存所有权交给系统；失败才GlobalFree。A_NET_AGAIN 实际是同意再战，邻近“不同意”的注释有误。

onlineLeaveText

online_ui.h : 361

作用与接口 按 why 和是否自己 mine 生成离开提示。

执行顺序 自己主动离开统一显示已通知；其他情况switch映射朋友离开、房主关房、退出、拒绝、取消、断线。

参数与局部量 why 为LeaveReason；mine 表示离开者是否本机。

onlineDisconnected

online_ui.h : 373

作用与接口 处理连接失败/丢失。

执行顺序 清连接及待确认；本来正在离开就回首页；尚未开局则退表单重试；已开局保留明确结束原因，否则补断线；未处理保存就询问，已处理则安排返回。

参数与局部量 不是重连功能：本局断线后只能保存复盘，不能继续原联网对局。

updateOnline

online_ui.h : 398

作用与接口 每帧处理联网计时、收消息、更新棋局和保存UI。

执行顺序 无关页面直接返回；处理自动返回/离开超时；连接中等就绪再start；循环take，分别处理延迟、ACK、WELCOME、事件、错误、STATE；合法STATE更新netState及game；新局清旧UI；新exitSerial回SEEN；终局每个round只启动一次保存。

参数与局部量 message 当前正文；value 解析LATENCY的流；closing 临时保留关闭窗口意图；state 临时解码快照；oldRound 原局号；leaveOnStart 记录等待室退出与开局竞争；seen 为SEEN输出流。

对照源码时留意

收到STATE时整体覆盖game；不要在UI保留未被服务端确认的“预测棋子”。一条普通STATE也会清netPending，不是靠每条请求独立ACK配对。

图 11　一次联网落子经过的对象



opponentProgress

online_ui.h : 511

作用与接口　给出对方保存/再战的进度文字。

执行顺序　身份无效或lost返回空；按again、done、saveStage顺序选提示。

参数与局部量　other=1-netRole 是对方座位。

drawOnlinePage

online_ui.h : 522

作用与接口　绘制联网菜单、表单、连接中、等待室和开局前关闭提示。

执行顺序　根据netStage分支；登记能用的按钮；等待室显示双方present/ready；退出询问时清buttons只保留是/否。

参数与局部量　value 为端口说明缓冲；i 为两方索引。

对照源码时留意

绘图之外还登记按钮，实际执行仍在onlineAction。输入框本体由Win32控件绘制。

drawOnlineSidebar

online_ui.h : 566

作用与接口 绘制对局右侧联网控制区。

执行顺序 先画状态/颜色/步数/RTT；离开中只提示等待；archiveStage非0则画保存流程并return；否则登记悔棋/重开/认输/退出，再按N_SELF/N_PEER显示相应人的询问按钮。

参数与局部量 step 步数缓冲；latency 为RTT输出流；free 表示N_PLAY且非pending；mine 是否为owner；prompt 为请求类型对应文字。

对照源码时留意

不是每个玩家在N_SELF都能回答；只有owner能回答。N_PEER则只有另一方能回答，界面启用条件与apply权限一致。

读完自测

重开与再战为什么都有“等待对方”？因为双方都要先完成自己的保存决定。重开用done数组，正常终局再战用again数组；服务端不会因为一方先完成就清空另一方还在保存的旧盘。

CHAPTER 13

用七个练习，把源码变成自己的知识

每题先尝试回答，再对照后面的答案与定位。练习不需要修改原有存档。

练习	请自己推演
1. 规则边界	黑棋已六连，wins 是否算赢？终局后 undo 能否撤回？
2. 鼠标吸附	FIRST=(178,278)，SPACING=66；点击(640,740)落在哪里？距离该交点31像素时如何？
3. 人机悔棋	玩家执白，历史为黑、白、黑。按悔棋后保留多少手？谁下？
4. 文件防错	MOVES声称2，但文件只有一行MOVE；或者第一手stone=2，会在哪一步被拒绝？
5. 复盘时钟	play(1000)后，pause(2000)，play(8000)，下一手何时出现？
6. 过期AI	任务gen=10，玩家重开后全局generation=11；take拿到旧答案时会怎样？
7. 网络旧包	当前round=2，却收到MOVE 1 0 7 7；谁拒绝它，客户端如何恢复一致？

答案与对应函数

1. 六连满足 count>=5，算赢；终局后 undo 返回false。看 LocalGame::wins / undo。
2. (row,col)=(7,7)，cell=112；若偏移平方超过900，boardCell 返回-1。
3. lastHumanMove 返回下标1；while(size>1)撤掉黑方应答和白方那手，只留最早黑棋；轮到白。
4. 少一行会在读取或MOVE格式校验失败；第一手棋色为2会在 stone!=r.game.turn 处失败。
5. 默认下一手原定4000；2000暂停时还剩2000；8000恢复后deadline=10000。
6. updateAI 会回收旧线程，但gen不匹配，拒绝落子；需要时再发新任务。
7. NetState::apply 检查r!=round拒绝；服务器回 EVENT INVALID 和当前STATE，UI重新对齐。

五条调试路线：一次只盯一件事

想看懂的现象	推荐断点 / 观察变量
点击后没棋子	boardCell → humanCanMove → place；看cell、page、dialog、turn、result。联网再看netPending/phase。
AI似乎思考很久	updateAI → AIWorker::start/run/take；看thread、generation、completedDepth、deadline。避免把断点耗时当算法耗时。
保存按钮失败	saveGame/onlineSave → writeRecord → parseRecord；看名字、mode、replay、reason、实际路径。
复盘暂停后跳子	pause/play/tick；看now、remaining、deadline、cursor，而不是只看playing。
双方棋盘不同步	onlineMove → 服务端apply → netEncode → UI netDecode；逐处比较round、moves.size、turn。

读完后你应该能独立回答

为什么棋盘同时保留cells和moves？为什么读档不直接相信RESULT？为什么AI不直接改全局game？为什么TCP必须分帧？为什么退出还要SEEN和CLOSE_ACK？如果你能沿本书的调用路径回答这五题，并指出数据在哪个函数改变，就已经抓住这个项目的骨架。

学习时适合做的小修改

先在另存的练习副本中改toast时长、复盘间隔或按钮文字；再试着增加“显示坐标”的绘图提示。修改AI候选数、协议字段和文件版本之前，先能说明它们影响哪些调用者，并运行对应测试。

CHAPTER 附录 A

查阅表、容易误读的细节与验证记录

这里是回看源码时的速查区。项目自己的函数在下一附录可按名字查页码。

源码与注释有冲突时，读执行语句

位置	准确解释
main.cpp:123-125	hitButton 从0往后遍历，不是注释说的从后往前。
game_logic.h:33；ai_engine.h:62	方向数组用(行增量,列增量)；(1,0)向下，(0,1)向右。
network_protocol.h:82-83	MOVE中的a/b/c分别为手数/行/列，没有额外颜色字段。
network_protocol.h:90,95,100	ASK先到N_SELF，发起者自己确认；重开/悔棋再转N_PEER等对方。
online_ui.h:333-334	A_NET_AGAIN 是同意再战，不能按附近“不同意”的注释理解。
main.cpp:442-447	速度按钮在started为真后出现，不只在暂停时出现。
main.cpp:501	solidroundrect 画实心底板，这里没有传透明混合参数，不能只因注释就当半透明。
ai_engine.h:162-168	缓存最佳点在键命中即可拿来排序；缓存分数需要深度足够。
online_ui.h:109	按100.64.0.0/10地址段选择候选，不是严格识别Tailscale网卡。

调用 Windows / EGE 时，先认识这些接口

接口组	本工程中的职责
GetTickCount64 / Sleep	进程运行时用的单调毫秒计时与短暂休眠；与Unix秒、SYSTEMTIME不能直接混算。
time / localtime / GetLocalTime	记录开局Unix秒、把秒拆成当地日期、取得当前系统本地年月日时分秒。
CreateThread / WaitForSingleObject / CloseHandle	建线程、等线程/事件/进程状态、释放内核句柄；等待0为轮询，INFINITE一直等。

接口组	本工程中的职责
InterlockedExchange / CompareExchange	原子写/读取消与关闭标志，避免普通共享访问的同步问题。
Initialize/Enter/Leave/DeleteCriticalSection	创建/进入/离开/销毁临界区。被保护的是完整消息队列。
CreateProcessW / OpenProcess / TerminateProcess	创建服务进程、观察父进程、必要时强制终止本机启动的服务端。
CreateEventA / OpenEventA / SetEvent	房主与服务端用同名事件表示“监听已就绪”。
CreateWindowExW / ShowWindow / DestroyWindow	创建、显示隐藏、销毁原生输入框；HWND表示窗口句柄。
Get/SetWindowTextW / SetFocus / SendMessageW	取写输入框文字、切焦点、设置控件字体与输入长度限制。
PeekMessageW / TranslateMessage / DispatchMessageW	取出Windows消息并派发给回调，让输入框与窗口正常响应。
SetWindowLongPtrW / CallWindowProcW	设置窗口样式/替换回调；将非自定义消息转回旧处理函数。
CreateFileW / ReadFile / WriteFile	用宽路径打开并读写原始字节；必须检查实际读写数量。
GetFileSizeEx / FlushFileBuffers / MoveFileExW	检查文件长度、刷新写缓冲、把临时文件移为正式文件。
CreateDirectoryW / GetFileAttributesW	创建目录与检查类型/存在性。
FindFirst/NextFileW / FindClose / DeleteFileW	枚举列表并关闭搜索句柄；按指定路径删除存档。
WideCharToMultiByte / MultiByteToWideChar	在Windows宽字符串和UTF-8字节之间转换。

接口组	本工程中的职责
WSAStartup / WSACleanup / WSAGetLastError	初始化与收尾Winsock，读取最近套接字错误。
socket / bind / listen / accept / connect	创建网络端点；服务端绑定监听接入，客户端发起连接。
ioctlsocket / select / getsockopt	设非阻塞；检测连接可写/错误；获取连接错误状态。
send / recv / closesocket	传送/接收一部分字节；释放套接字，不能用CloseHandle代替。
htonl/ntohl / htons / inet_addr	32位网络序转换、16位端口网络序转换、IPv4文本转地址。
GetAdaptersAddresses / CoCreateGuid	枚举本机IPv4与生成房主身份口令的GUID。
Open/Empty/Set/CloseClipboard	打开、清空、填入Unicode文本、关闭剪贴板。
GlobalAlloc/Lock/Unlock/Free	准备剪贴板所需的可移动内存，成功交付后由系统持有。
newimage / getimage / delimage	EGE图片分配、加载、释放；PIMAGE是图像对象指针。
putimage / putimage_withalpha / putimage_alphablend	贴图、带透明通道的缩放贴图、用指定透明度绘制悬停预览。
font/text内部的setfont、outtextxy等	设置绘图字体与颜色、测量字宽高、输出文字。
solidroundrect / circle / setlinewidth	圆角底板、棋子标记圈、描边宽度。
mousemsg / getmouse / mousepos / kbhit / getkey	读鼠标事件、查询位置、清键盘缓冲；原生输入框文字靠Windows消息处理。
initgraph / render模式 / delay_fps / closegraph	初始化图形窗口、手动刷新模式、限帧与退出清理。

外部结构体：只看实际使用到的字段

结构体及字段	作用
LOGFONTW f：lfHeight / lfQuality / lfFaceName	字体像素高度、抗锯齿质量、字体名称；先ZeroMemory避免未填字段有随机值。
SYSTEMTIME now或t：wYear/wMonth/wDay/wHour/wMinute/wSecond/wMilliseconds	生成可读时间名；wMilliseconds只在文件名避让时使用。
WIN32_FIND_DATAW info：dwFileAttributes / cFileName	判断是否目录或重解析点；取找到的文件名。
LARGE_INTEGER size：QuadPart	64位文件字节长度。
std::tm* t：tm_year/tm_mon/tm_mday/tm_hour/tm_min/tm_sec	年从1900计，月份0..11；其他字段可直接格式化。
sockaddr_in：sin_family / sin_port / sin_addr.s_addr	AF_INET表示IPv4；端口与地址用网络字节序；INADDR_ANY绑定所有本机网卡。
timeval wait：tv_sec / tv_usec	select超时用秒和微秒；{0,10000}表示10毫秒。
STARTUPINFOW si：cb / dwFlags / wShowWindow	结构大小、启用显示配置标记、隐藏窗口；PROCESS_INFORMATION pi的hProcess/hThread须关闭。
GUID id：Data1/Data2/Data3/Data4[8]	组成唯一标识的多个数字字段，被格式化成房主token。
IP_ADAPTER_ADDRESSES：Next / OperStatus / FirstUnicastAddress	网卡链表下一项、启用状态、单播地址链表；每个地址节点用Next及Address.lpSockaddr读取IPv4。
MSG message：message	判断是否WM_QUIT；其他字段交给TranslateMessage和DispatchMessage。
EGE mouse_msg m：x/y/wheel及is_*方法	鼠标坐标、滚轮方向；判断左键、按下、松开、滚动。

常见短变量，不必每见一次都重新猜

i/j/k 通常是局部循环下标，含义随容器变化；r/c 或 row/col 通常是行列，但 parseRecord 的 r 是记录对象。s 有时是字符串，有时是状态或棋色，必须看类型。out/o 是输出字符串流，in/input 是输入流，extra 用来检测本来不应该出现的尾随字段。bool ok/loaded/alive 只是当前步骤的成功判断，并不自动证明整个系统状态正确。

std::min/max 用于夹取范围；sort 按比较函数排列；find 检查容器成员，string::find 找子串；substr 取一段；erase 删除一段；resize 改长度；empty 判空；size 取数量；c_str/data 取交给C接口的缓冲地址。memset/ZeroMemory 清内存，memcpy复制定长字节；sprintf/swprintf 格式化窄/宽字符串；sscanf拆IPv4；atoi/strtoul/wcstol把文本转整数。

编译与测试：本书实际验证到了哪里

预处理入口：NOMINMAX 防止 Windows 宏 min/max 干扰 std::min/max；_WIN32_WINNT=0x0601（客户端还设 WINVER=0x0601）声明所需的Windows API级别。头文件的 #ifndef/#define/#endif 防止同一编译单元重复包含；不是“运行时只执行一次”。先包含 network_transport.h，使 winsock2.h 位于一般 windows.h 之前，避免旧网络声明冲突。

本次将当前源码编译到 tmp/pdfs 的隔离输出，不覆盖 Gomoku3 下原有 exe 或棋谱。编译器使用 build.ps1 指定的 D:/Dev-Cpp/MinGW64/bin/g++.exe；C++11、64位、O2、Wall、Wextra、GBK输入。客户端链接 EGE 与Windows图形/网络相关库，服务端链接 ws2_32。两个可执行文件均成功生成，编译无诊断输出。

重新编译并运行的测试	本次结果
tests/core_tests.cpp	通过：合法性、回合、悔棋、四方向、长连、终局锁、清盘、平局。
tests/stage_tests.cpp	通过：v1/v2与AI元数据、坏档、覆盖写、复盘时序、三档AI、线程取消。
tests/network_tests.cpp	通过：权威状态、旧/重复消息、弹窗锁、重开保存屏障、再战与旧版联网复盘。
tests/network_v2_tests.cpp	通过：单双步悔棋、拒绝、认输、保存阶段、拒绝/取消原因、v4零步认输复盘及v3兼容。

这一次AI测试局面中：一般档完成4层，约265毫秒；困难档完成5层，约2000毫秒。它们是当时机器与局面的观测值，不是固定性能承诺。测试在 tmp/pdfs 中运行，写盘测试不会碰原用户棋谱。

验证边界：本次没有把旧exe当作当前源码的证明，也没有重新执行真实双窗口UI或跨电脑网络测试。上述网络测试验证的是状态/协议逻辑；本书对窗口与TCP循环的解释另依据当前源码逐分支阅读。编译与测试通过不等于不存在任何边界缺陷。

源码版本指纹与定位

以下SHA-256短前缀用于辨认此次文件。完整指纹保存在同目录的“源码核对清单.json”；源码修改后函数行号可能移动。

文件 / 行数	SHA-256前16位
game_logic.h / 74	ae15e57f3ea834c5
main.cpp / 740	ae26dd16c31adabd
match_metadata.h / 50	3f7097b686a84ef1
record_store.h / 229	812e71f2f3945b6c
replay_logic.h / 59	e4150e7eabd7376e
ai_engine.h / 298	187649cdd31d3bce
network_protocol.h / 166	a31656a6b4579e0c
network_transport.h / 187	c01717f861afec99
server_main.cpp / 175	5b9254e398707ace
online_ui.h / 630	bd0304a2b3146f6b

CHAPTER 附录 B

函数检索表

按名字排序；右栏是本 PDF 的讲解页。构造函数和回调也列入索引。

函数 / 位置	PDF 页
act [main.cpp:262]	17
addButton [main.cpp:109]	14
AIEntry::AIEntry [ai_engine.h:18]	26
AISearch::AISearch [ai_engine.h:203]	27
AISearch::candidates [ai_engine.h:94]	29
AISearch::choose [ai_engine.h:217]	31
AISearch::evaluate [ai_engine.h:139]	29
AISearch::pointScore [ai_engine.h:61]	28
AISearch::put [ai_engine.h:50]	28
AISearch::random [ai_engine.h:44]	27
AISearch::search [ai_engine.h:153]	30
AISearch::stopped [ai_engine.h:37]	27
AIWorker::AIWorker [ai_engine.h:275]	33
AIWorker::cancel [ai_engine.h:282]	33
AIWorker::run [ai_engine.h:277]	33
AIWorker::shutdown [ai_engine.h:296]	34
AIWorker::start [ai_engine.h:284]	34
AIWorker::take [ai_engine.h:291]	34

函数 / 位置	PDF 页
beginName [main.cpp:219]	16
boardCell [main.cpp:360]	14
cancelAI [main.cpp:72]	34
contains [main.cpp:116]	14
drawButtons [main.cpp:129]	14
drawGame [main.cpp:370]	17
drawOnlinePage [online_ui.h:522]	57
drawOnlineSidebar [online_ui.h:566]	58
encodeRecord [record_store.h:163]	21
font [main.cpp:98]	15
fromUtf8 [record_store.h:35]	20
GameRecord::GameRecord [record_store.h:21]	19
gameWindowProc [main.cpp:257]	13
hideName [main.cpp:145]	15
hitButton [main.cpp:123]	14
humanCanMove [main.cpp:67]	13
lastHumanMove [main.cpp:75]	34
loadEntry [main.cpp:229]	16
loadImage [main.cpp:589]	18
localAddresses [online_ui.h:85]	52
LocalGame::LocalGame [game_logic.h:21]	8

函数 / 位置	PDF 页
LocalGame::place [game_logic.h:54]	9
LocalGame::reset [game_logic.h:24]	8
LocalGame::undo [game_logic.h:66]	10
LocalGame::wins [game_logic.h:32]	9
mainMenu [main.cpp:147]	15
main（客户端） [main.cpp:595]	13
main（服务端） [server_main.cpp:17]	47
matchTime [match_metadata.h:40]	20
NetClient::entry [network_transport.h:97]	43
NetClient::NetClient [network_transport.h:162]	43
NetClient::push [network_transport.h:95]	43
NetClient::run [network_transport.h:99]	44
NetClient::sendMessage [network_transport.h:179]	44
NetClient::start [network_transport.h:173]	44
NetClient::stop [network_transport.h:167]	44
NetClient::take [network_transport.h:181]	45
NetClient::~NetClient [network_transport.h:164]	43
netDecode [network_protocol.h:146]	40
netEncode [network_protocol.h:134]	40
NetSocket::attach [network_transport.h:29]	42
NetSocket::close [network_transport.h:26]	41

函数 / 位置	PDF 页
NetSocket::NetSocket [network_transport.h:24]	41
NetSocket::pump [network_transport.h:43]	43
NetSocket::queue [network_transport.h:31]	42
NetState::apply [network_protocol.h:68]	39
NetState::closeRoom [network_protocol.h:49]	38
NetState::finish [network_protocol.h:41]	38
NetState::lastMove [network_protocol.h:43]	38
NetState::leave [network_protocol.h:58]	39
NetState::NetState [network_protocol.h:29]	37
NetState::start [network_protocol.h:35]	38
newGame [main.cpp:149]	15
numberLine [record_store.h:62]	20
onlineAction [online_ui.h:299]	56
onlineAnswer [online_ui.h:149]	54
onlineArchiveDone [online_ui.h:178]	54
onlineAsk [online_ui.h:143]	53
onlineCanMove [online_ui.h:120]	53
onlineCommand [online_ui.h:132]	53
onlineConnect [online_ui.h:236]	55
onlineDeparture [online_ui.h:159]	54
onlineDisconnected [online_ui.h:373]	56

函数 / 位置	PDF 页
onlineEnter [online_ui.h:75]	52
onlineForms [online_ui.h:46]	52
onlineHome [online_ui.h:68]	52
onlineLeaveText [online_ui.h:361]	56
onlineLobbyLeave [online_ui.h:293]	55
onlineMove [online_ui.h:136]	53
onlineName [online_ui.h:230]	55
onlineRequestHome [online_ui.h:167]	54
onlineSave [online_ui.h:200]	55
onlineSaveStatus [online_ui.h:155]	54
onlineSend [online_ui.h:127]	53
onlineStop [online_ui.h:54]	52
openList [main.cpp:159]	16
opponentProgress [online_ui.h:511]	57
parseRecord [record_store.h:77]	21
Peer::Peer [server_main.cpp:55]	47
readRecord [record_store.h:146]	22
reasonText [match_metadata.h:27]	20
recordDirectories [record_store.h:47]	20
render [main.cpp:524]	18
ReplayPlayer::open [replay_logic.h:21]	23

函数 / 位置	PDF 页
ReplayPlayer::pause [replay_logic.h:36]	24
ReplayPlayer::play [replay_logic.h:41]	24
ReplayPlayer::ReplayPlayer [replay_logic.h:19]	23
ReplayPlayer::seek [replay_logic.h:29]	24
ReplayPlayer::speed [replay_logic.h:46]	24
ReplayPlayer::tick [replay_logic.h:52]	25
savedSuccessfully [main.cpp:215]	16
saveGame [main.cpp:187]	16
text [main.cpp:104]	15
toast [main.cpp:94]	15
toUtf8 [record_store.h:25]	20
updateAI [main.cpp:337]	35
updateOnline [online_ui.h:398]	56
validResult [match_metadata.h:15]	19
writeRecord [record_store.h:189]	22